百科荣创
BECSTRONG

EXPERIMENT GUIDE BOOK | 实验指导书

百科荣创（北京）科技发展有限公司

Python 基础教程

前 言

百科荣创成立于 2003 年，是国内领先的新一代信息技术产业高素质工程技术与技术技能人才培养解决方案提供商，设有北京总部、济南分部（产教融合服务中心、企业公共实习实践基地）、深圳分部（研发制造中心）、成都分部（筹），国家级高新技术企业与双软认证企业，国内电子、通信、嵌入式、物联网、人工智能等电子信息领域教育头部企业，面向全国高校、职业院校提供专业共建、人才联合培养、实验实训装备、课程开发、教材联合开发、学生实习实践与就业等产品与产教融合服务。公司自成立以来，坚持秉承“产品就是人品！做企业就是做人！”的经营理念和“来源于教育，回报于教育”的企业精神，汇聚了一大批电子、通信、嵌入式、物联网、人工智能方向的优秀人才，以培养智能化时代电子信息大类复合型技术技能人才为宗旨，致力于为客户提供最满意的产品和服务，力争成为中国工程教育与职业教育领域的产教融合型领军企业。

十多年来，公司产品行销全国各省、市、自治区、成为众“双一流”高校以及职业院校采购同类产品的首选，公司产品主要涉及职教工程实训和高教实验研究两大领域，涵盖了电子、通信、物联网、移动互联与智能控制、机器人及人工智能等专业学科方向系列实验实训教学及创新设备，用户已遍及清华大学、北京大学、北京理工大学、北京科技大学、天津大学、重庆大学、东北大学、四川大学、深圳职业技术学院、天津电子信息学院、上海电子信息职业技术学院、北京电子科技职业学院等全国各大知名院校。

为了给您提供更加优质的服务，公司设有 400 开头的免费电话，在使用过程中，如果您有任何问题需要咨询或者需要服务，请与本公司相关部门联系，我们定当竭诚为您服务。服务电话：400-067-7899。

欢迎您在使用的过程中为我们提出宝贵的意见和建议。

目 录

第 1 章 Python 基础实验 ---------- 1
1.1 Python 基础实验 ---------- 1
1.2 Python 注释实验 ---------- 4
1.3 Python 语句格式实验 ---------- 5
1.4 Python 行和缩进实验 ---------- 6
1.5 Python 标识符实验 ---------- 7
1.6 Python 关键字实验 ---------- 8
1.7 Python 输入与输出实验 ---------- 9
1.8 Python 数据类型实验 ---------- 10
1.9 Python 字符串实验 ---------- 13
1.10 Python 列表实验 ---------- 15
1.11 Python 元组实验 ---------- 17
1.12 Python 字典实验 ---------- 20
1.13 Python 数据类型转换实验 ---------- 22
1.14 Python 算术运算符实验 ---------- 28
1.15 Python 条件语句实验 ---------- 32
1.16 Python 循环语句实验 ---------- 34
第 2 章 Python 高级实验 ---------- 38
2.1 Python 函数实验 ---------- 38
2.2 Python 变量实验 ---------- 40
2.3 Python 面向对象实验 ---------- 41
2.4 Python 继承实验 ---------- 45
2.5 Python 多态实验 ---------- 48
2.6 Python 异常处理实验 ---------- 51
2.7 Python 模块实验实验 ---------- 53
2.8 Python 包实验 ---------- 54
2.9 Python 文件 I/O 实验 ---------- 55

2.10 Python 进程与线程实验 ----------60
第 3 章 Python 项目实验 ----------65
3.1 九九乘法表实验 ----------65
3.2 闰年判断实验 ----------67
3.3 数字组合实验 ----------69
3.4 Python web 服务器实验 ----------71
3.5 Python 爬虫实验 ----------74
3.6 Python 飞机大战实验 ----------78

第1章 Python 基础实验

1.1 Python 基础实验

1.1.1 实验目的

1. 了解 Python 编程语言基本概念。
2. 掌握 Python 基础语法知识。

1.1.2 实验内容

掌握 Python 语法结构，包括标识符、关键词、缩进等基础语法。

1.1.3 实验原理及步骤

1. Hello World

Python 编译方式分为交互式编译和脚本式编译。交互式编程不需要创建脚本文件，是通过 Python 解释器的交互模式进来编写代码。在 linux 上你只需要在命令行中输入 python3 命令即可启动交互式编程，如下：

```
bkrc@bkrc:~$ python3
Python 3.7.3 (default, Apr  3 2019, 05:39:12)
>>>
```

紧随在“>>>”敲入 python 代码，注意在交互模式下输入的代码是立即执行的，并且把每一行 python 代码的结果自动打印出来。退出键入 exit()。

脚本式编程是指把 python 代码写入通常以 py 为后缀的文件中，在终端命令行中调用 Python 解释器来加载执行文件中的 python 代码。还记的 cat 命令吗？让我们使用 cat 命令来创建一个 sayHello.py 文件，写一个 python 版的 hello world。

```
bkrc@bkrc:~$ cat > sayHello.py
print("Say Hello!")
print('Hello World!’)
```

执行 sayHello.py 脚本

```
bkrc@bkrc:~$ python3 sayHello.py
Say Hello!
Hello World!
```

仔细观察你会发现，在 python 中使用英文双引号（””）和英文单引号（‘’）均可表示字符串，是的它们在表示字符串方面没有什么不同。

2. 标识符

在 Python 里，标识符由字母、数字、下划线组成。在 Python 中，所有标识符可以包括英文、数字以及下划线(_)，但不能以数字开头。Python 中的标识符是严格区分大小写的。

以下划线开头的标识符是有特殊意义的。以单下划线开头_foo 的代表不能直接访问的类属性，需通过类提供的接口进行访问，不能用 from xxx import *导入。

以双下划线开头的__foo 代表类的私有成员。

以双下划线开头和结尾的__foo__代表 Python 里特殊方法专用的标识，如__init__()代表类的构造函数。

Python 允许一行书写多个语句，方法是用分号“;”分开，示例如下:

```
>>> 1+2;print("say hello");2+4
3
say hello
6
```

3. Python 关键字

表 1.1.1 显示了 Python 的关键字，这些关键字是保留字，不能用做标识符。所有 Python 的关键字只包含小写字母。

表 1.1.1 关键字

and	exec	not	assert	finally	or	break
for	pass	class	from	print	continue	global
raise	def	if	return	del	import	try
elif	in	while	else	is	with	except
lambda	yield	-	-	-	-	-

表中可能列举的不够完备，你可以使用如下命令来查看 python 中的所有关键字

```
>>> import keyword
>>> keyword.kwlist
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

4. 变量定义

Python 中变量定义不需要类型声明，这将由类型推断自动完成。每个定义的变量

在内存中创建，其中包括变量的标识，名称和数据这些信息。

等号（=）用来给变量赋值。等号（=）运算符左边是一个变量名，右边是存储在变

量中的值。示例如下：

```
# -*- coding: UTF-8 -*-
counter = 10    # 整型变量
miles = 10.0        # 浮点型
name = "san"    # 字符串
print (counter)
print (miles)
print (name)
```

运行结果如下所示：

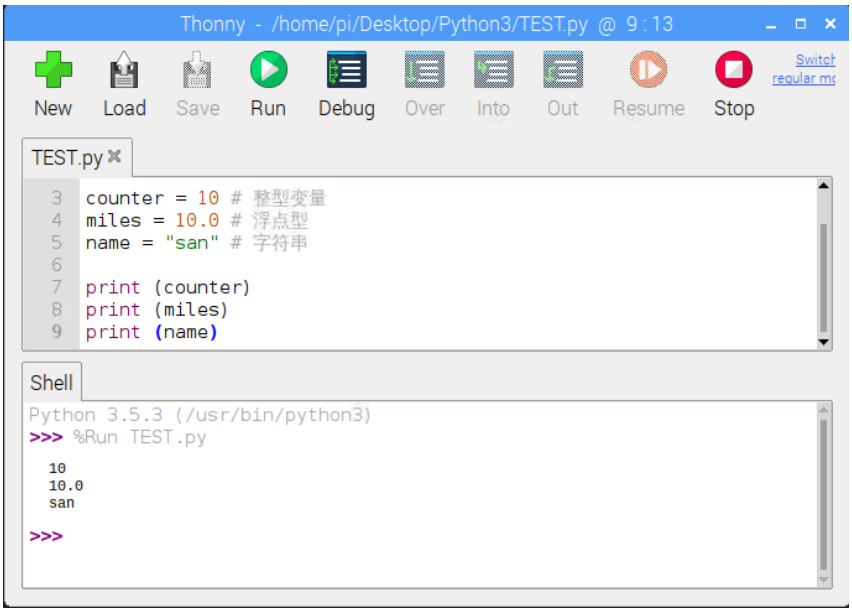

图 1.1.1 变量定义示例代码

1. 1. 4 思考题

Python 的语法结构和 C 语言及 java 有哪些不同之处？

1.2 Python 注释实验

1.2.1 实验目的

1. 了解 Python 两种注释方法。
2. 掌握 Python 两种注释方法实验。

1.2.2 实验内容

掌握 Python 单行注释与多行注释的写法与使用方式。

1.2.3 实验原理及步骤

1.单行注释

Python 中使用#表示单行注释。单行注释可以作为单独的一行放在被注释代码行之上，也可以放在语句或表达式之后。

当单行注释作为单独的一行放在被注释代码行之上时，为了保证代码的可读性，建议在#后面添加一个空格，再添加注释内容。

当单行注释放在语句或表达式之后时，同样为了保证代码的可读性，建议注释和语句（或注释和表达式）之间至少要有两个空格。

```
print("这是#注释")
#print("这是#注释")
```

2.多行注释

当注释内容过多，导致一行无法显示时，就可以使用多行注释。Python 中使用三个单引号或三个双引号表示多行注释。

```
print("这是"""单引号注释")
'''print("这是"""单引号注释")

'''

print("这是双引号注释")
"""
print("这是双引号注释")
"""
```

1.2.4 思考题

Python 的注释与其他高级语言有什么区别吗？

1.3 Python 语句格式实验

1.3.1 实验目的

1. 了解 Python 语句格式写法。

1.3.2 实验内容

掌握 Python 语句格式写法。

1.3.3 实验原理及步骤

Python 语句写法是每行是一个语句，没有用分号结尾；当多行语句时，则需要在每一行的语句后面加上分号。

```
# -*- coding:utf-8 -*-
#或者
#coding:utf-8

print('hello world')
print('hello python');print('hello world')  #多行语句在一行的时候需要在语句之间加上分号
result = 123 + 234 + \
        234 + 324 + \
        567
print(result)
```

1.3.4 思考题

Python 的语句格式与其他高级语言有什么区别吗？

1.4 Python 行和缩进实验

1.4.1 实验目的

1. 了解 Python 行与缩进。

1.4.2 实验内容

掌握 Python 行与缩进的写法与使用。

1.4.3 实验原理及步骤

python 最具特色的就是使用缩进来表示代码块，不需要代码块{};缩进的空格数是可变的，但是同一个代码块的语句必须包含相同的缩进的空格是可变的，但是同一个代码块的语句必须包含相同的缩进空格数。

```
print("行的正确与错误写法")
'''
print('hello')
 print('hello')
'''
print("缩进"+'\n')#换行符
age=19
if age >= 18:
    print('成年')
else:
    print('未成年')
print('判断结束')
```

1.4.4 思考题

Python 的行与缩进与其他高级语言有什么区别吗？

1.5 Python 标识符实验

1.5.1 实验目的

1. 了解 Python 标识符。

1.5.2 实验内容

掌握 Python 标识符的作用与写法。

1.5.3 实验原理及步骤

Python 标识符是用来给变量、函数、类取名字

1. 名字里面只能由***字母、数字、下划线(_)***组成
2. 不能以***数字***开头

注意：python3.x 中标识符中可以出现中文，但是实际开发中不可以这么使用，标示符由字母、下划线和数字组成，且数字不能开头。区分大小写的。例如，andy 和 Andy 是不同的标识符。不能使用关键字。为了规范命名标识符，关于标识符的命名提以下建议。

常量名：使用大写的单个单词或由下画线连接的多个单词。

模块名、函数名： 使用小写的单个单词或由下画线连接的多个单词。

类名：使用大写字母开头的单个或多个单词。

```
str1=[1,2,3,4,5]#str1 就是标识符，也是变量
print(str1)
```

1.5.4 思考题

Python 的标识符与其他高级语言是一样的吗？

1.6 Python 关键字实验

1.6.1 实验目的

1. 了解 Python 关键字。

1.6.2 实验内容

掌握 Python 有哪些关键字。

1.6.3 实验原理及步骤

在 Python 中，具有特殊功能的标识符称为关键字。关键字是 Python 语言自己已经使用的了，不允许开发者自己定义和关键字相同名字的标识符

```
>>> import keyword
>>> keyword.kwlist
['False', 'None', 'True', 'and', 'as', 'assert', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

需要注意的是，由于 Python 是严格区分大小写的，保留字也不例外。所以可以说 if 是保留字，但 IF 就不是保留字。

在实际开发中，如果使用 Python 中的保留字作为标识符，则解释器会提示“invalid syntax” 的错误信息。

1.6.4 思考题

Python 的关键字与其他高级语言是一样的吗？

1.7 Python 输入与输出实验

1.7.1 实验目的

1. 了解 Python 输入与输出函数。

1.7.2 实验内容

掌握 Python 输入与输出函数并使用。

1.7.3 实验原理及步骤

Python 两种输出值的方式: 表达式语句和 print() 函数。

第三种方式是使用文件对象的 write()方法，标准输出文件可以用 sys.stdout 引用。如果希望输出的形式更加多样，可以使用 str.format() 函数来格式化输出值。

如果希望将输出的值转成字符串，可以使用 repr() 或 str() 函数来实现。

str()： 函数返回一个用户易读的表达形式。

repr()： 产生一个解释器易读的表达形式。

Python 提供了 input() 内置函数从标准输入读入一行文本，默认的标准输入是键盘。

```
'''
python 两种输出方式：表达式语句和 print()
第三种方式就是使用文件对象的 write()

str.formot() 格式化输出
repr() 输出值转成字符串
str() 输出值转成字符串
'''
s='hello,runoob'
print(type(str(s)))
print(str(s)+'\n')
print(type(repr(s)))
print(repr(s))
s1=input('这是输入函数')

print(s1)
```

1.7.4 思考题

Python 的输入与输出函数与其他高级语言是一样的吗？

1.8 Python 数据类型实验

1.8.1 实验目的

1. 了解 Python 有哪些数据类型。

1.8.2 实验内容

掌握 Python 数据类型的使用

1.8.3 实验原理及步骤

数据类型是每种编程语言必备属性，只有给数据赋予明确的数据类型，计算机才能对数据进行处理运算，因此，正确使用数据类型是十分必要的，不同的语言，数据类型类似，但具体表示方法有所不同，以下是 Python 编程常用的数据类型。

1. 数字类型

Python 数字类型主要包括 int（整型）、long（长整型）和 float（浮点型），但是在 Python3 中就不再有 long 类型了。

int（整型）

在 32 位机器上，整数的位数是 32 位，取值范围是 -231~231-1，即 -2147483648~214748364；在 64 位系统上，整数的位数为 64 位，取值范围为-263~263-1，即 9223372036854775808~9223372036854775807。

long（长整型）

Python 长整型没有指定位宽，但是由于机器内存有限，使用长的长整数数值也不可能无限大。

float（浮点型）

浮点型也就是带有小数点的数，其精度和机器有关。

complex（复数）

Python 还支持复数，复数由实数部分和虚数部分构成，可以用 a + bj,或者 complex(a,b) 表示， 复数的实部 a 和虚部 b 都是浮点型。

2. 字符串

在 Python 中，加了引号的字符都被认为是字符串，其声明有三种方式，分别是：单引号、双引号和三引号；Python 中的字符串有两种数据类型，分别是 str 类型和

unicode 类型，str 类型采用的 ASCII 编码，无法表示中文，unicode 类型采用 unicode 编码，能够表示任意字符，包括中文和其他语言。

3. 布尔型

和其他编程语言一样，Python 布尔类型也是用于逻辑运算，有两个值：True（真）和 False（假）。

4. 列表

列表是 Python 中使用最频繁的数据类型，集合中可以放任何数据类型，可对集合进行创建、查找、切片、增加、修改、删除、循环和排序操作。

5. 元组

元组和列表一样，也是一种序列，与列表不同的是，元组是不可修改的，元组用”()”标识，内部元素用逗号隔开。

6. 字典

字典是一种键值对的集合，是除列表以外 Python 之中最灵活的内置数据结构类型，列表是有序的对象集合，字典是无序的对象集合。

7. 集合

集合是一个无序的、不重复的数据组合，它的主要作用有两个，分别是去重和关系测试。

```
"""
1、按经验将不同的变量存储不同的数据类型
2、验证这些数据到底是什么类型--检测数据类型--type(数据) 函数
"""
#int -- 整型
num1 = 1
#float -- 浮点型，就是小数
num2 = 1.1
print(type(num1))
print(type(num2))
#str -- 字符串 特点是数据都是要带英文状态下的引号（单双引号都行）
a = "hello,world!"
print(type(a))
#bool -- 布尔型，通常判断使用，布尔类型有两个取值 True 和 False
b = True
print(type(b))
```

```
#list -- 列表
c = [10,20,30]
print(type(c))

#tuple -- 元组
d = (10,20,30)
print(type(d))
#set -- 集合
e = {10,20,30}
print(type(e))

#dict -- 字典
f = {'name':'NAIVA','age':18} #键值对
print(type(f))
```

1.8.4 思考题

Python 的数据类型与其他高级语言是一样的吗？

1.9 Python 字符串实验

1.9.1 实验目的

1. 了解 Python 字符串基本概念。
2. 掌握 Python 字符串的使用方法。

1.9.2 实验内容

掌握 Python 字符串的定义、索引等基础用法。

1.9.3 实验原理及步骤

在 C/C++等经典语言中，我们知道定义字符串，需要使用双引号（""）把字符内容扩起来，在 Python 中也是一样,在 python 中可以这样来定义一个字符串。

```
S1 = “Say Hello”
```

实际上，python 在字符串上玩法比较多，在 python 中还可以使用单引号（‘‘）来定义字符串。这和使用双引号定义的字符串没有什么不同。

```
s1 = "Say Hello"
s2 = 'Say Hello'
print(type(s1), type(s2))

# == 值比较 is 是内存比较
print(s1 == s2)
print(s1 is s2)
print(id(s1), id(s2))
```

另外在 python 中还可以使用三对单/双引号来定义字符串，使用三个引号括起来的字符，是一段多行注释。一般用于 py 文件开头用于说明这个 py 文件作用，public Document。一般用来保持原来定义的格式。一般用来写公共函数的注释。

```
# 保留原格式打印
print(""""Say


        ....
            Hello""")
```

和 Java 一样，在 Python 中字符串也是常量，如下代码验证了这一点

```
>>> s1 = "Say Hello"
>>> id(s1)
3061182856
```

```
>>> s1 = s1 + "!"
>>> s1
'Say Hello!'
>>> id(s1)
3060505480
```

从结果可以看到，在对字符串进行拼接后，s1 这个字符串对象内存地址发生了变化。这表明 python 字符串是保存在内存常量池中。

Python 的字符串类型是 str，在内存中以 Unicode 表示，一个字符对应若干个字节。如果要在网络上传输，或者保存到磁盘上，就需要 str 把变为以字节为单位的 bytes。

Python 对 bytes 类型的数据用带前缀 b 的单引号或双引号表示：

```
>>> x=b"ABC"
>>> x
b'ABC'
```

要注意区分'ABC'和 b'ABC'，前者是 str，后者虽然内容显示得和前者一样，但 bytes 的每个字符都只占用一个字节。

str 在内存中以 Unicode 表示，可通过 encode()方法可以编码为指定的 bytes。

```
>>> ‘ABC’.encode(‘ascii’)
b’ABC’
>>> ‘中文’.eccode('utf-8') b'\xe4\xb8\xad\xe6\x96\x87'
```

如果从网络或磁盘上读取字节流，读到的数据类型是 bytes。要把 bytes 变为 str，就需要用方法 decode()。

```
>>> b’ABC’.decode(‘utf-8’)
 ‘ABC’
>>> ‘\xe4\xb8\xad\xe6\x96\x87’.decode(‘utf-8’)
 ‘中文’
```

str 类型的对象有很多实用的方法，如 encode，decode，具体使用 dir(str)，help（str）查看。

1.9.4 思考题

Python 的字符串索引方法有哪些？

1.10 Python 列表实验

1.10.1 实验目的

1. 了解 Python 列表基本概念。
2. 掌握 Python 列表的使用方法。

1.10.2 实验内容

掌握 Python 列表的创建、读写、删除等基础用法。

1.10.3 实验原理

序列是 Python 中最基本的数据结构。序列中的每个元素都分配一个数字索引，第一个索引是 0，第二个索引是 1，以此类推。Python 有 6 个序列的内置类型，但最常见的是列表和元组。序列都可以进行的操作包括索引，切片，加，乘，检查成员。此外，Python 已经内置确定序列的长度以及确定最大和最小的元素的方法。

列表是最常用的 Python 数据类型，它可以作为一个方括号内的逗号分隔值出现。列表的数据项不需要具有相同的类型

Python 列表函数&方法如表 1.10.1 所示：

表 1.10.1 列表函数

序列	函数
1	len(list) 返回列表元素个数
2	max(list) 返回列表元素最大值
3	min(list) 返回列表元素最小值
4	list(seq) 将元组转换成列表

1.10.4 实验步骤

创建一个列表，只要把逗号分隔的不同的数据项使用方括号括起来即可。如下所示：

```
myList = [1, 2, 'a', 'b']
# 访问列表
print("第一个元素", myList[0])
print("第二个元素", myList[1])
print("第三个元素", myList[2])
print("第四个元素", myList[3])
```

```
# 更新列表
# 更新把列表中的 a 元素更新为 3
myList[2] = 3
print("更新后的列表",myList)
# 删除列表中元素
# 删除列表中的 b 元素
del myList[3]
print("删除 b 元素",myList)
# 过去列表元素个数
print("列表元素个数",len(myList))
```

运行结果：

```
第一个元素 1
第二个元素 2
第三个元素 a
第四个元素 b
更新后的列表 [1, 2, 3, 'b']
删除 b 元素 [1, 2, 3]
```

1.10.5 思考题

Python 的列表操作方法和 C 语言的数组操作方法有什么区别？

1.11 Python 元组实验

1.11.1 实验目的

1. 了解 Python 元组基本概念。
2. 掌握 Python 元组的基本使用方法。

1.11.2 实验内容

掌握 Python 元组的创建、索引、删除等基础用法。

1.11.3 实验原理及步骤

Python 字典是另一种可变容器模型，且可存储任意类型对象，如字符串、数字、元组等其他容器模型。

Python 的元组与列表类似，不同之处在于元组的元素不能修改。元组使用小括号，列表使用方括号。

1. 创建元组

元组创建很简单，只需要在括号中添加元素，并使用逗号隔开即可。

```
tup1 = ('physics', 'chemistry', 1997, 2000)
tup2 = (1, 2, 3, 4, 5 )
tup3 = "a", "b", "c", "d"
```

元组与字符串类似，下标索引从 0 开始，可以进行截取，组合等。

2. 访问元组

元组可以使用下标索引来访问元组中的值。

```
#!/usr/bin/python
tup1 = ('physics', 'chemistry', 1997, 2000)
tup2 = (1, 2, 3, 4, 5, 6, 7 )
print("tup1[0]: ", tup1[0])
print("tup2[1:5]: ", tup2[1:5])
```

程序运行结果如下：

```
tup1[0]:  physics
tup2[1:5]:  (2, 3, 4, 5)
```

3. 修改元组

元组中的元素值是不允许修改的，但可以对元组进行连接组合。:

```
tup1 = (12, 34.56)
```

```
tup2 = ('abc', 'xyz')
# 以下修改元组元素操作是非法的。
# tup1[0] = 100 # 创建一个新的元组
tup3 = tup1 + tup2
print(tup3)
```

程序运行结果如下：

```
(12, 34.56, 'abc', 'xyz')
```

4. 删除原则

元组中的元素值是不允许删除的，但可以使用 del 语句来删除整个元组，

```
tup = ('physics', 'chemistry', 1997, 2000)
print tup
del tup
print "After deleting tup : "
# print tup
```

5. 元组运算符

与字符串一样，元组之间可以使用 + 号和 * 号进行运算。也即是元组可以组合和复制，运算后会生成一个新的元组，如表 1.11.1 所示：

表 1.11.1 元组运算示例

Python 表达式	结果	描述
len((1, 2, 3))	3	计算元素个数
(1, 2, 3) + (4, 5, 6)	(1, 2, 3, 4, 5, 6)	连接
('Hi!',) * 4	('Hi!', 'Hi!', 'Hi!', 'Hi!')	复制
3 in (1, 2, 3)	True	元素是否存在
for x in (1, 2, 3): print x,	1 2 3	迭代

6. 元组索引

因为元组也是一个序列，所以我们可以访问元组中的指定位置的元素，也可以截取索引中的一段元素。

```
L = ('spam', 'Spam', 'SPAM!')
print(L[2])
print(L[-2])
print(L[1:])
```

程序运行结果如下：

```
SPAM!
Spam
('Spam', 'SPAM!')
```

1.11.4 思考题

Python 的元组操作方法和 C 语言的数组操作方法有什么区别？

1.12 Python 字典实验

1.12.1 实验目的

1. 了解 Python 字典基本概念。
2. 掌握 Python 字典的使用方法。

1.12.2 实验内容

掌握 Python 字典的创建、索引、删除等基础用法。

1.12.3 实验原理及步骤

Python 字典是另一种可变容器模型，且可存储任意类型对象，如字符串、数字、元组等其他容器模型。

（1）创建字典

字典的每个键值(key=>value)对用冒号(:)分割，每个对之间用逗号(,)分割，整个字典包括在花括号({})中，格式如下所示：

```
d = {key1 : value1, key2 : value2 }
```

键必须是唯一的，但值则不必。值可以取任何数据类型，但键必须是不可变的，如字符串，数字或元组。

创建字典实例：

```
my_dict = {'Name': 'bkrc', 'Age': 10, 'Site': 'beijing'}
```

（2）访问字典

把相应的键放入([])中即可访问键值所对应的值，如果访问字典里没有的键值程序将会出错。访问字典实例：

```
>>> print("名称：", my_dict['Name'])
输出：
名称： bkrc
```

实例：

```
# 访问字典
print("名称：", my_dict['Name'])
print("年龄：", my_dict['Age'])
print("地点：", my_dict['Site'])

# 修改字典
```

```
my_dict["Age"] = 20
print("修改后的年龄：", my_dict['Age'])

# 删除字典
del my_dict["Age"]
print(my_dict)
```

程序运行结果如图 1.12.1 字典实例运行结果所示：

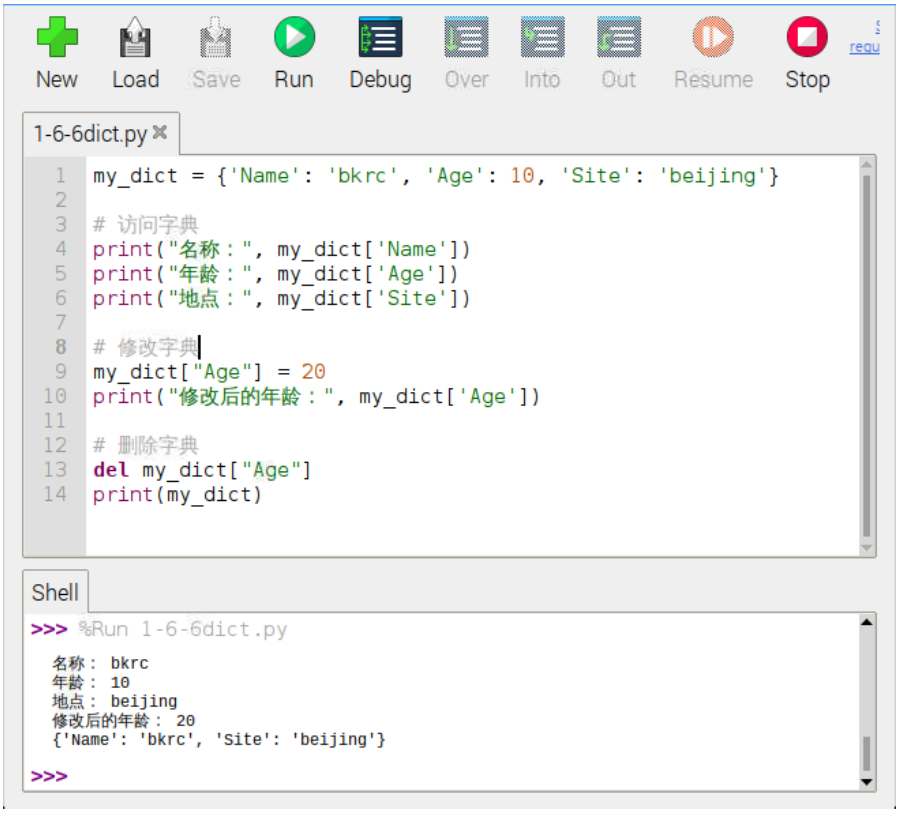

图 1.12.1 字典实例运行结果

1. 12. 4 思考题

Python 的字典操作方法和 java 的 map 类有什么区别？

1.13 Python 数据类型转换实验

1.13.1 实验目的

1. 了解 Python 数据类型转换。

1.13.2 实验内容

掌握 Python 数据之间如何进行数据类型转换的。

1.13.3 实验原理及步骤

Python 实际上并没有把一个东西从 y 中类型“转换”成另一种类型，它只是由原来的东西创建一个新的东西，而且这个新东西正是你想要的类型。下面给出一些函数，他们可以从一种类型转换为另一种类型：

float()从一个字符串或整数创建一个新的浮点数（小数）；

int()从一个字符串或浮点数创建一个新的整数（取整数）；

str()从一个数（可以是任何其他类型）创建一个新的字符串。

float()、int()、str()后面有小括号，因为他们不是 Python 关键字---他们只是 Python 的内置函数。

Python 还提供了函数 type()，它可以明确的告诉我们变量的类型。

```
# 定义 float 类型变量
f1 = 1.12
f2 = 1.99
# 定义 bool 类型变量
b1 = True
b2 = False
# 定义 str 类型变量
s1 = '5'
# 使用 int()把 float、bool、str 类型数据转换为 int 类型
num1 = int(f1)
num2 = int(f2)
num3 = int(b1)
num4 = int(b2)
num5 = int(s1)
# 输出结果
print(f'num1 = {num1}')
print(f'num2 = {num2}')
```

```
print(f'num3 = {num3}')
print(f'num4 = {num4}')
print(f'num5 = {num5}')


# 定义 int 类型变量
num1 = 256
num2 = 1024
# 定义 bool 类型变量
b1 = True
b2 = False
# 定义 str 类型变量
s1 = '5'
# 使用 float()把 int、bool、str 类型数据转换为 float 类型
f1 = float(num1)
f2 = float(num2)
f3 = float(b1)
f4 = float(b2)
f5 = float(s1)
# 输出结果
print(f'f1 = {f1}')
print(f'f2 = {f2}')
print(f'f3 = {f3}')
print(f'f4 = {f4}')
print(f'f5 = {f5}')


# 定义 int 类型变量
num1 = 0
num2 = -2
num3 = 2
# 定义 float 类型变量
f1 = 0.0
f2 = -2.0
f3 = 2.0
# 定义 str 类型变量
s1 = ''
s2 = '0'
s3 = 'a'
# 使用 bool()把 int、float、str 类型数据转换为 bool 类型
b1 = bool(num1)
```

```
b2 = bool(num2)
b3 = bool(num3)
b4 = bool(f1)
b5 = bool(f2)
b6 = bool(f3)
b7 = bool(s1)
b8 = bool(s2)
b9 = bool(s3)
# 输出结果
print(f'b1 = {b1}')
print(f'b2 = {b2}')
print(f'b3 = {b3}')
print(f'b4 = {b4}')
print(f'b5 = {b5}')
print(f'b6 = {b6}')
print(f'b7 = {b7}')
print(f'b8 = {b8}')
print(f'b9 = {b9}')


# 定义 int 类型变量
num1 = 256
num2 = 1024
# 定义 float 类型变量
f1 = 2.56
f2 = 1.024
# 定义 bool 类型变量
b1 = False
b2 = True
# 定义 list 类型变量
l1 = [1, 2, 3, 4, 5]
l2 = ['a', 'b', 'c', 'd']
# 定义 tuple 类型变量
t1 = (1, 2, 3, 4, 5)
t2 = ('a', 'b', 'c', 'd')
# 定义 set 类型变量
ss1 = {1, 2, 3, 4, 5}
ss2 = {'a', 'b', 'c', 'd'}
# 定义 dict 类型变量
d1 = {'张珊': 18, '李四': 15, '王五': 20}
d2 = {'name': '张珊', 'age':18, 'sex': '女'}
```

```
# 将 int、float、bool、list、tuple、set、dict 类型的数据转换为 str 类型
s1 = str(num1)
s2 = str(num2)
s3 = str(f1)
s4 = str(f2)
s5 = str(b1)
s6 = str(b2)
s7 = str(l1)
s8 = str(l2)
s9 = str(t1)
s10 = str(t2)
s11 = str(ss1)
s12 = str(ss2)
s13 = str(d1)
s14 = str(d2)
# 输出结果
print(f's1 = {s1}')
print(f's2 = {s2}')
print(f's3 = {s3}')
print(f's4 = {s4}')
print(f's5 = {s5}')
print(f's6 = {s6}')
print(f's7 = {s7}')
print(f's8 = {s8}')
print(f's9 = {s9}')
print(f's10 = {s10}')
print(f's11 = {s11}')
print(f's12 = {s12}')
print(f's13 = {s13}')
print(f's14 = {s14}')
# 输出数据类型
print(type(s1))
print(type(s2))
print(type(s3))
print(type(s4))
print(type(s5))
print(type(s6))
print(type(s7))
print(type(s8))
print(type(s9))
print(type(s10))
```

```
print(type(s11))
print(type(s12))
print(type(s13))
print(type(s14))



# 定义 tuple 类型变量
t1 = (1, 2, 3, 4, 5)
t2 = ('a', 'b', 'c', 'd')
# 定义 set 类型变量
ss1 = {1, 2, 3, 4, 5}
ss2 = {'a', 'b', 'c', 'd'}
# 定义 dict 类型变量
d1 = {'张珊': 18, '李四': 15, '王五': 20}
d2 = {'name': '张珊', 'age':18, 'sex': '女'}
# 将 tuple、set、dict 类型数据转换为 list 类型
ll1 = list(t1)
ll2 = list(t2)
ll3 = list(ss1)
ll4 = list(ss2)
ll5 = list(d1)
ll6 = list(d2)
# 输出结果
print(f'll1 = {ll1}')
print(f'll2 = {ll2}')
print(f'll3 = {ll3}')
print(f'll4 = {ll4}')
print(f'll5 = {ll5}')
print(f'll6 = {ll6}')


# 定义 list 类型变量
l1 = [1, 2, 3, 4, 5]
l2 = ['a', 'b', 'c', 'd']
# 定义 set 类型变量
ss1 = {1, 2, 3, 4, 5}
ss2 = {'a', 'b', 'c', 'd'}
# 定义 dict 类型变量
d1 = {'张珊': 18, '李四': 15, '王五': 20}
d2 = {'name': '张珊', 'age':18, 'sex': '女'}
```

```
# 将 list、set、dict 类型数据转换为 tuple 类型
tt1 = tuple(l1)
tt2 = tuple(l2)
tt3 = tuple(ss1)
tt4 = tuple(ss2)
tt5 = tuple(d1)
tt6 = tuple(d2)
# 输出结果
print(f'tt1 = {tt1}')
print(f'tt2 = {tt2}')
print(f'tt3 = {tt3}')
print(f'tt4 = {tt4}')
print(f'tt5 = {tt5}')
print(f'tt6 = {tt6}')

# 定义 list 类型变量
l1 = [1, 2, 3, 4, 5]
l2 = ['a', 'b', 'c', 'd']
# 定义 tuple 类型变量
t1 = (1, 2, 3, 4, 5)
t2 = ('a', 'b', 'c', 'd')
# 定义 dict 类型变量
d1 = {'张珊': 18, '李四': 15, '王五': 20}
d2 = {'name': '张珊', 'age':18, 'sex': '女'}
# 将 list、tuple、dict 类型数据转换为 set 类型
sss1 = set(l1)
sss2 = set(l2)
sss3 = set(t1)
sss4 = set(t2)
sss5 = set(d1)
sss6 = set(d2)
# 输出结果
print(f'sss1 = {sss1}')
print(f'sss2 = {sss2}')
print(f'sss3 = {sss3}')
print(f'sss4 = {sss4}')
print(f'sss5 = {sss5}')
print(f'sss6 = {sss6}')
```

1.13.4 思考题

Python 的的数据类型转换与其他高级语言有什么区别的吗？

1.14 Python 算术运算符实验

1.14.1 实验目的

1. 了解 Python 运算符基本概念。
2. 掌握算术运算符、比较运算符、逻辑运算符使用方法。

1.14.2 实验内容

掌握 Python 运算符，包括算术运算符、比较运算符、逻辑运算符等基础用法。

1.14.3 实验原理及步骤

Python 语言支持以下类型的运算符:算术运算符：比较（关系）运算符、赋值运算符、逻辑运算符、位运算符、成员运算符、身份运算符、运算符优先级。

1. 算术运算符

常见的算术运算符如下表所示：

表 1.14.1 算术运算符

运算符	描述
+	加-两个对象相加
-	减一得到负数或是一个数减去另一个数
*	乘 - 两个数相乘或是返回一个被重复若干次的字符串
/	除 - x 除以 y
%	取模 - 返回除法的余数
*	幂 - 返回 x 的 y 次幂
//	取整除 - 返回商的整数部分（向下取整）

实例：

```
# -*- coding: UTF-8 -*-
a = 25
b = 10
c = 0
c = a + b
print ("{} + {} = {}".format(a,b,c))
c = a - b
print ("{} - {} = {}".format(a,b,c))
c = a * b
print ("{} * {} = {}".format(a,b,c))
```

```
c = a / b
print ("{} / {} = {}".format(a,b,c))
c = a % b
print ("{} % {} = {}".format(a,b,c))
# 修改变量 a 、b 、c
a = 2
b = 3
c = a**b
print ("{} ** {} = {}".format(a,b,c))
a = 10
b = 5
c = a//b
print ("{} // {} = {}".format(a,b,c))
```

输出结果：

```
25 + 10 = 35
25 - 10 = 15
25 * 10 = 250
25 / 10 = 2.5
25 % 10 = 5
2 ** 3 = 8
10 // 5 = 2
```

2. 比较运算符

常见的比较运算符如下表所示：

表 1.14.2 比较运算符

运算符	示例	描述
= =	x == y	等于 - 比较 x 与 y 是否相等
!=	x！ = y	不等于 - 比较 x 与 y 是否不相等
<	x < y	小于 - 比较 x 是否小于 y
>	x > y	大于 - 比较 x 是否大于 y
<=	x <= y	小于等于 - 比较 x 是否小于等于 y
>=	x >= y	大于等于 - 比较 x 是否大于等于 y

实例：

```
x = 1
y = 2
print("x==y:",x==y)
print("x!=y:",x!=y)
print("x>y:" ,x>y)
```

```
print("x<y:" ,x<y)
print("x>=:" ,x>=y)
print("x<=:y",x<=y)
```

运行结果：

```
x==y: False
x!=y: True
x>y: False
x<y: True
x>=: False
x<=:y True
```

3. 逻辑运算符

常见的逻辑运算符如下表所示：

表 1.14.3 逻辑运算符

运算符	示例	描述
and	x and y	布尔"与" - 如果 x 为 False，x and y 返回 False，否则它返回 y 的计算值。
or	x or y	布尔"或" - 如果 x 是 True，它返回 x 的值，否则它返回 y 的计算值。
not	not x	布尔"非" - 如果 x 为 True，返回 False 。如果 x 为 False，它返回 True。

实例如下：

```
a = True
b = False
if a and b:
    print("1 - 变量 a 和 b 都为 true")
else:
    print("1 - 变量 a 和 b 至少有一个不为 true")

if a or b:
    print("2 - 变量 a 和 b 至少有一个为 True")
else:
    print("2 - 变量 a 和 b 都为 false")

if not a:
    print("3 - 变量 a 为 false")
else:
    print("3 - 变量 a 为 true")
```

运行结果：

```
1 - a 和 b 至少有一个不为 true
2 - 变量 a 和 b 至少有一个为 True
3 - 变量 a 为 true
```

1.14.4 思考题

掌握了算术运算符、比较运算符、逻辑运算法的基本使用方法，那么他们的优先级是什么？

1.15 Python 条件语句实验

1.15.1 实验目的

1. 了解 Python 条件基本概念。
2. 掌握 Python 条件语句的基本语法。
3. 掌握条件语句的使用方法。

1.15.2 实验内容

掌握 Python 条件语句的基础用法。

1.15.3 实验原理及步骤

1. 条件语句

Python 条件语句是通过一条或多条语句的执行结果（True 或者 False）来决定执行的代码块。所有非 None 的对象在条件测试时是 True，None 是 False，0 是 False，非零是 True。下图显示了条件语句的执行过程:

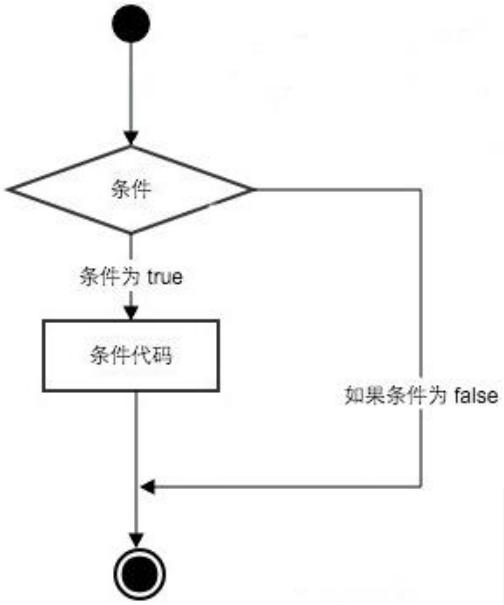

图 1.15.1 条件控制流程图

Python 编程中 if 语句用于控制程序的执行，基本形式为：

```
if 判断条件：
    执行语句……
elif 判断条件:
```

```
    执行语句……
else：
    执行语句…
```

其中"判断条件"成立时（非零），则执行后面的语句，而执行内容可以多行，以缩进来区分表示同一范围。else 为可选语句，当需要在条件不成立时执行内容则可以执行相关语句，具体例子如下：

```
a = 1
b = 2
if a > b:
    print("a 大于 b")
elif a<b:
    print("a 小于 b")
else:
    print("a 等于 b")
```

不断修改变量 a,b 的来熟悉条件控制语句的运作方式。

1.15.4 思考题

Python 的条件语句和 C 语言有什么区别？

1.16 Python 循环语句实验

1.16.1 实验目的

1. 了解 Python 循环基本概念。
2. 掌握 Python 循环语句的基本语法。
3. 掌握循环语句的使用方法。

1.16.2 实验内容

掌握 Python 循环语句的基础用法。

1.16.3 实验原理及步骤

1. 循环控制语句

程序在一般情况下是按顺序执行的。编程语言提供了各种控制结构，允许更复杂的执行路径。循环语句允许执行一个语句或语句组多次，循环控制语句则可以更改语句执行的顺序。下图所示是在大多数编程语言中的循环语句的一般形式：

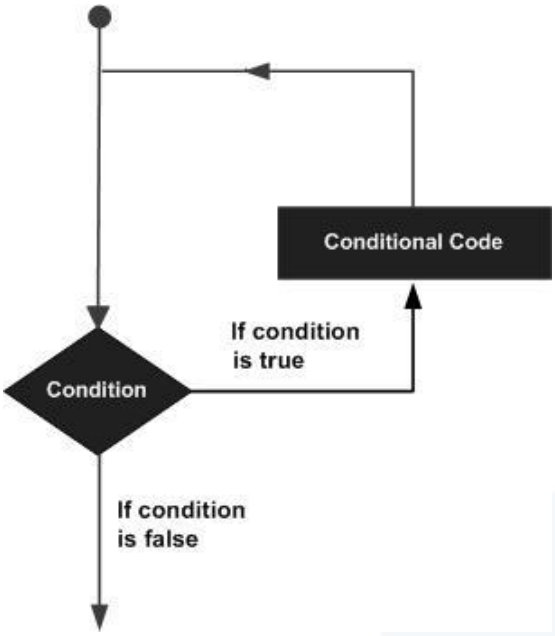

图 1.16.1 循环结构

Python 提供了循环语句有：for 循环和 while 循环。循环控制语句有 break、continue、pass。

（1）While 循环语句：

Python 编程中 while 语句用于循环执行程序，即在某条件下，循环执行某段程序，以处理需要重复处理的相同任务。其基本形式为：

```
while 判断条件：
   执行语句……
当判断条件假 false 时，循环结束。
```

实例：每隔 1 秒中程序循环打印一次，在程序中 import 导入延时所需要的包。通过 sleep()控制循环延时时间，如果 i>100 这个条件成立则跳出整个 while 循环。

```
import time
i = 0
while True:
   print(i)
   i = i+1
   time.sleep(1)
   if I > 100:
       break
```

（2）for 循环语句

Python for 循环可以遍历任何序列的项目，如一个列表或者一个字符串。for 循环的一般格式如下：

```
for <variable> in <sequence>:
   <statements>
else:
   <statements>
```

实例 1：range()为生成数列函数

```
for i in range(1,5):
   print(i)
```

实例 2：字符串是可迭代对象

```
>>> for c in "Say Hello":
...     print(c)
...
S
a
y

H
e
```

```
l
l
o
>>>
```

（3）break 控制循环语句

break 语句可以跳出 for 和 while 的循环体。如果你从 for 或 while 循环中终止，任何对应的循环 else 块将不执行。实例如下：

```
import time
i = 0
while True:
    print(i)
    i = i+1
    time.sleep(1)
    if i == 5:
        break
print("over")
```

当 i 等于 5 时，退出循环。运行结果如下：

```
0
1
2
3
4
Over
```

（4）continue 控制循环语句

continue 语句被用来告诉 Python 跳过当前循环块中的剩余语句，然后继续进行下一轮循环。如下实例:

```
for i in range(1,5):
    if i == 3:
        continue
    print(i)
```

当程序中变量 i 等于 3 时，执行 continue 跳过下面打印语句，进入下一次循环。执行效果如下：

```
1
2
```

```
4
```

（5）pass 控制循环语句

Python pass 是空语句，是为了保持程序结构的完整性。pass 不做任何事情，一般用作占位语句，如下实例:

```
for i in range(1,5):
    if i == 3:
        pass
    print(i)
```

当程序中变量 i 等于 3 时，执行 pass 语句，不做任何事情。

1.16.4 思考题

Python 的循环语句和 C 语言有什么区别？

第2章 Python 高级实验

2.1 Python 函数实验

2.1.1 实验目的

1. 了解 Python 函数基本语法。
2. 掌握 Python 函数的使用方法。

2.1.2 实验内容

掌握 Python 函数的基础语法。

2.1.3 实验原理及步骤

函数是组织好的，可重复使用的，用来实现单一，或相关联功能的代码段。函数能提高应用的模块性，和代码的重复利用率。你已经知道 Python 提供了许多内建函数，比如 print()。但你也可以自己创建函数，这被叫做用户自定义函数。

1．定义一个函数

定义一个由自己想要功能的函数需要遵守以下规则：

（1）函数代码块以 def 关键词开头，后接函数标识符名称和圆括号()。

（2）任何传入参数和自变量必须放在圆括号中。圆括号之间可以用于定义参数。

（3）函数的第一行语句可以选择性地使用文档字符串—用于存放函数说明。

（4）函数内容以冒号起始，并且缩进。

（5）return [表达式] 结束函数，选择性地返回一个值给调用方。不代表达式的 return 相当于返回 None。

2.语法

```
def functionname( parameters ):
  do something
  return [expression]
```

3．实例

下面是一个简单的实例，函数 add()将传入的参数 x，y 相加并返回相加后的结果，在主函数中调用 add()方法，输出返回的结果。

```
def add (x, y):
  z = x + y
  return z
```

```
# 调用 add()方法
z = add(1,2)
print(z)
```

运行结果：

```
3
```

2.1.4 思考题

Python 的函数基本语法与 C 语言的函数语法有哪些不同？

2.2 Python 变量实验

2.2.1 实验目的

1. 了解 Python 变量的基本概念与使用。

2.2.2 实验内容

掌握 Python 变量的用法。

2.2.3 实验原理及步骤

1. 定义变量

语法：变量名 = 数据

变量名： 程序员自己命名。

要求：标识符、不能是关键字、

规范：见名知义 。

满足 PEP8 的命名规范。（所有字母都小写，所有单词之间用下划线隔开）不使用系统提供的类型名、函数名和模块名。

```
"""
1、定义变量
   语法：变量名 = 值
2、使用变量
3、变量的特点
"""
#定义变量：存储数据 NAIVA
my_name = 'NAIVA'  #定义变量
print(my_name)    #使用变量

#定义变量：存储数据
hisName = '百科荣创'
print(hisName)
```

2.2.4 思考题

Python 的变量与其他高级语言是一样的吗？

2.3 Python 面向对象实验

2.3.1 实验目的

1. 了解 Python 面向对象的基本概念与使用方法。

2.3.2 实验内容

掌握 Python 面向对象的概念与编程原理。

2.3.3 实验原理及步骤

1. 面向对象

面向对象：将变量与函数绑定到一起，分类进行封装，每个程序只要负责分配给自己的分类，这样能够更快速的开发程序，减少了重复代码。

（1）面向过程编程的关注点在于怎么做？

把完成某一个需求的所有步骤从头到尾逐步实现，根据开发需求，将某些功能独立的代码封装成一个又一个函数，最后完成的代码，就是顺序地调用 不同的函数

特点:

注重步骤与过程，不注重职责分工；

如果需求复杂，代码会变得很复杂；

开发复杂项目，没有固定的套路，开发难度很大；

面向对象编程：

面向对象编程（Object Oriented Programming，OOP，面向对象程序设计）和面向过程编程，是两种不同的编程方式。相比较函数，面向对象是更大的封装，根据职责在 一个对象中封装多个方法

（2）面向对象编程的关注点在于谁来做？

在完成某一个需求前，首先确定职责，也就是要做的事情（方法）；

根据 职责 确定不同的 对象，在对象内部封装不同的方法（多个）；

最后完成的代码，就是顺序地调用不同对象的相应方法。

特点:

注重对象和职责，不同的对象承担不同的职责。

更加适合应对复杂的需求变化，是专门应对复杂项目开发，提供的固定套路。

2. 类和对象

（1）类

类是对一群具有相同 特征 或者 行为 的事物的一个统称，是抽象的，不能直接使用。

特征其实就是一个变量，在类里我们称之为属性。

行为其实就是一个函数，在类里我们称之为方法。

类其实就是由 属性 和 方法 组成的一个抽象概念。

（2）对象（实例）

对象是由类创建出来的一个具体存在，可以直接使用。由哪一个类创建出来的对象，就拥有在哪一个类中定义的属性和方法。在开发中，应该先有类，在类里定义好属性和行为，再根据类来创建对象。

3. 类和对象的关系

类是模板，对象是根据类这个模板创建出来的，应该先有类，再有对象。使用同一个类，能够创建出很多对象。类中定义了什么属性和方法，对象中就有什么属性和方法。不同对象对应的属性值也会不同。

```
import sys

# 我们目前所学习的对象都是 Python 内置类的对象
a = "10" # 创建一个 str 类的实例
b = str("hello") # 创建一个 str 类的实例
print(a, type(a))
print(b, type(b))

'''
对象的创建流程:
   1.声明变量
   2.在内存中创建一个新对象
   3.将对象的 id 赋值给变量
'''

# id()是 python 的内置函数，用于返回对象的身份，即对象的内存地址。

c = a
print(id(a))
```

```
print(id(c))

'''
变量引用区别
  Python 缓存了整数和字符串，因此每个对象在内存中只存有一份，引用所指对象就是相同的，即使使用赋值语句，也只是创造新的引用，而不是对象本身；
  Python 没有缓存列表及其他对象，可以由多个相同的对象，可以使用赋值语句创建出新的对象。
'''
# 通过 is 进行引用所指判断，is 是用来判断两个引用所指的对象是否相同。
print(a is c)

d = 1
e = 1
print(d is e)

d = "good"
e = "good"
print(d is e)

d = "hello world"
e = "hello world"
print(d is e)

d = []
e = []
print(d is e)

'''
引用计数
  变量：通过变量指针引用对象，变量指针指向具体对象的内存空间，取对象的值。
  对象：类型已知，每个对象都包含一个头部信息（头部信息：类型标识符和引用计数器）
  sys.getrefcount()：每个对象都有指向该对象的引用总数
'''
print(sys.getrefcount(b))
print(sys.getrefcount("hello"))
# ps:当使用某个引用作为参数，传递给 getrefcount()时，参数实际上创建了一个临时的引用。因此，getrefcount()所得到的结果，会比期望的多 1。
print(b , type(b))
print(b , type(type(b)))
print(b , type(type(type(b))))
```

```
print(b , id(b))
print(b , id(type(b)))
print(b , id(type(type(b))))
print(b , id(type(type(type(b)))))
print(b , id(type(type(type(type(b))))))
print(a , id(type(type(type(type(a))))))
```

2.3.4 思考题

Python 的面向对象与其他高级语言是一样的吗？

2.4 Python 继承实验

2.4.1 实验目的

1. 了解 Python 继承的基本概念与使用。

2.4.2 实验内容

掌握 Python 继承的概念与用法。

2.4.3 实验原理及步骤

1. 什么是继承？

继承就是让类和类之间转变为父子关系，子类可以直接访问(调用)父类的静态属性和方法。

在 python 中，新建的类可以继承一个或多个父类，父类又可称为基类或超类，新建的类称为派生类或子类。

2. 继承的规则

子类继承父类的成员变量和成员方法；

子类不继承父类的构造方法，能够继承父类的析构方法；

子类不能删除父类的成员，但可以重定义父类成员；

子类可以增加自己的成员。

3. 继承的作用

实现代码（功能）的重用，降低代码冗余；

增强软件可扩充性；

提高软件的维护性。

4. 继承与抽象的概念

面向对象中的两个重要概念：抽象和分类。

抽象和分类是人们认识世界的基本方法。

5. 派生类

在父类的基础上产生子类，产生的子类就叫做派生类；

父类里没有的方法，在子类中有了，这样的方法就叫做派生方法；

父类里有，子类也有的方法，就叫做方法的重写（就是把父类里的方法重写了)。

```
# 继承
class People:
    # 构造方法
    def __init__(self, n, a):
        self.name = n
        self.age = a

    def info(self):
        print("我叫 %s ， 今年 %d 岁" % (self.name, self.age))


'''
单继承
    父类中的所有方法都会被子类继承，包括特殊方法，也可以重写特殊方法
'''


class Student(People):
    def __init__(self, n, a, g):
        super().__init__(n, a)
        self.grade = g

    # 覆写父类的方法
    def info(self):
        print("我叫 %s ， 今年 %d 岁， 正在读 %s" % (self.name, self.age, self.grade))


s = Student("Ken", 19, "大一")
s.info()

'''
多重继承
    在 Python 中是支持多重继承的，可以在类名的()后边添加多个类，来实现多重继承
    需要注意圆括号中父类的顺序，若是父类中有相同的方法名，而在子类使用时未指定，
python 从左至右搜索 即方法在子类中未找到时，从左到右查找父类中是否包含方法。
'''


class DeepLearn:
    def aims(self):
        print("我的目标是学习深度学习")
```

```
class BigDate:
    def aims(self):
        print("我的目标是学习大数据")


class AIStudent(People, DeepLearn, BigDate):
    def __init__(self, n, a, g):
        People.__init__(n, a)
        self.grade = g

    # 覆写父类的方法
    def info(self):
        print("我叫 %s ， 今年 %d 岁， 正在读 %s" % (self.name, self.age, self.grade))


ais = AIStudent("Jim", 20, "大二")
ais.info()
ais.aims()
```

2.4.4 思考题

Python 的继承与其他高级语言是一样的吗？

2.5 Python 多态实验

2.5.1 实验目的

1. 了解 Python 多态的基本概念与使用。

2.5.2 实验内容

掌握 Python 多态的概念与使用。

2.5.3 实验原理及步骤

1. 多态概念：

面向对象的三大特征：封装，继承，多态；

封装：根据职责将属性和方法封装到一个抽象的类中；

继承：实现代码的重用，相同的代码不需要重复的编写；

多态:(以封装和继承为前提)不同的子类调用相同的方法，产生不同的结果。

2. 类属性 and 类方法

(1)类是一个特殊的对象--类对象

在程序运行时，类对象在内存中只有一份，通过他可以创建出很多个对象实例，除了封装实例的属性和方法外，类对象还可以拥有自己的属性和方法。

(2)类属性：就是给类对象定义的属性

类属性不会用于记录具体的对象特征；

使用赋值语句在 class 关键字下方可以定义类属性。

(3)类方法：就是针对类对象定义的方法，在类方法内部就可以直接访问类属性或调用类。

```
# 多态
# 定义两个类
class A:
    def __init__(self, name):
        self._name = name

    @property
    def name(self):
        return self._name
```

```
    @name.setter
    def name(self, name):
        self._name = name


class B:
    def __init__(self, name):
        self._name = name

    def __len__(self):
        return 10

    @property
    def name(self):
        return self._name

    @name.setter
    def name(self, name):
        self._name = name


class C:
    pass


a = A("孙悟空")
b = B("猪八戒")
c = C()


def say_hello(obj):
    print("你好 %s" % obj.name)


say_hello(a)
say_hello(b)
# say_hello(c)

'''
类型检查
    issubclass(obj, sub) - 判断一个类是另一个类的子类或者子孙类。
```

```
    isinstance(obj, Class) - 如果 obj 是 Class 类的实例对象或者是一个 Class 子类的实例对象则返回 true。
    注意：类型检查违反了多态原则，这会导致程序的适应性非常差，注意使用场合。
    '''


def say_hello(obj):
    if isinstance(obj, A):
        print("你好 %s" % obj.name)


say_hello(a)
say_hello(b)
```

2.5.4 思考题

Python 的多态与其他高级语言是一样的吗？

2.6 Python 异常处理实验

2.6.1 实验目的

1. 了解 Python 异常处理的基本概念与使用。

2.6.2 实验内容

掌握 Python 异常处理的概念与使用的方法。

2.6.3 实验原理及步骤

1. 什么是异常？

异常即是一个事件，该事件会在程序执行过程中发生，影响了程序的正常执行。一般情况下，在 Python 无法正常处理程序时就会发生一个异常。异常是 Python 对象，表示一个错误。当 Python 脚本发生异常时我们需要捕获处理它，否则程序会终止执行。

2. 异常处理

捕捉异常可以使用 try/except 语句。try/except 语句用来检测 try 语句块中的错误，从而让 except 语句捕获异常信息并处理。如果你不想在异常发生时结束你的程序，只需在 try 里捕获它。语法：以下为简单的 try....except...else 的语法。

try 的工作原理是，当开始一个 try 语句后，python 就在当前程序的上下文中作标记，这样当异常出现时就可以回到这里，try 子句先执行，接下来会发生什么依赖于执行时是否出现异常。

如果当 try 后的语句执行时发生异常，python 就跳回到 try 并执行第一个匹配该异常的 except 子句，异常处理完毕，控制流就通过整个 try 语句（除非在处理异常时又引发新的异常）。

如果在 try 后的语句里发生了异常，却没有匹配的 except 子句，异常将被递交到上层的 try，或者到程序的最上层（这样将结束程序，并打印默认的出错信息）。

如果在 try 子句执行时没有发生异常，python 将执行 else 语句后的语句（如果有 else 的话），然后控制流通过整个 try 语句。

```
class MyError(Exception):
    def __init__(self, value):
        self.value = value

    def __str__(self):
```

```
        return repr("111" + self.value)


def add(a, b):
    # 如果 a 和 b 中有负数，就向调用处抛出异常
    if a < 0 or b < 0:
        # raise 用于向外部抛出异常，后边可以跟一个异常类，或异常类的实例
        # raise Exception
        # 抛出异常的目的，告诉调用者这里调用时出现问题，希望你自己处理一下
        # raise Exception('两个参数中不能有负数！')
        raise MyError('自定义的异常')

        # 也可以通过 if else 来代替异常的处理
        # return None
    r = a + b
    return r


print(add(-123, 456))

'''
如果自己编写的某个模块需要抛出多个不同但相关的异常，
可以先创建一个基类，然后创建多个派生类分别表示不同的异常。
'''
class HerError(MyError):
    pass

class HisError(MyError):
    pass
```

2.6.4 思考题

Python 的异常处理与其他高级语言是一样的吗？

2.7 Python 模块实验实验

2.7.1 实验目的

1. 了解 Python 模块操作的基本语法。
2. 掌握 Python 对模块的导入及使用基本操作方法。

2.7.2 实验内容

掌握 Python 对模块的基本操作方法。

2.7.3 实验原理及步骤

Python 模块(Module)，其实是一个 Python 文件，以.py 结尾，包含了 Python 对象定义和 Python 语句。模块让你能够有逻辑地组织你的 Python 代码段。把相关的代码分配到一个模块里能让你的代码更好用，更易懂。模块能定义函数，类和变量，模块里也能包含可执行的代码。

1．创建模块

下例是创建一个简单的模块 addModule.py：里面包含一个加法运算的方法

```
def add (x, y):
  z = x + y
  return z
```

2．import 语句

模块创建好之后使用 import 语句来引入模块，语法如下：

```
import module1，module2，...moduleN
```

例如下面引用刚刚创建的 addModule.py 模块。注意：一个模块只会被导入一次，不管执行了多少次 import。使用示例如下：

```
import addModule

z = addModule.add(1,2)
print(z)
```

输出结果：

```
3
```

2.7.4 思考题

怎么引用 Python 模块？

2.8 Python 包实验

2.8.1 实验目的

1. 了解 Python 包的基本概念

2.8.2 实验内容

掌握 Python 包的概念与用法。

2.8.3 实验原理及步骤

对于一个需要实际应用的模块而言，往往会具有很多程序单元，包括变量、函数和类等，如果将整个模块的所有内容都定义在同一个 Python 源文件中，这个文件将会变得非常庞大，显然并不利于模块化开发。

1. 什么是包

为了更好地管理多个模块源文件，Python 提供了包的概念。那么问题来了，什么是包呢？从物理上看，包就是一个文件夹，在该文件夹下包含了一个 __init__.py 文件，该文件夹可用于包含多个模块源文件；从逻辑上看，包的本质依然是模块。

2. 定义包

掌握了包是什么之后，接下来学习如何定义包。定义包更简单，主要有两步：

创建一个文件夹，该文件夹的名字就是该包的包名；

在该文件夹内添加一个 __init__.py 文件即可。

```
print('请注意，每一个包目录下面都会有一个__init__.py 的文件，这个文件是必须存在的，'+'\n'
    '否则，Python 就把这个目录当成普通目录(文件夹)，'+'\n'
    '而不是一个包。init.py 可以是空文件，也可以有 Python 代码，'+'\n'
    '因为__init__.py 本身就是一个模块，而它的模块名就是对应包的名字。')
```

2.8.4 思考题

Python 的包与其他高级语言有什么区别吗？

2.9 Python 文件 I/O 实验

2.9.1 实验目的

1. 了解 Python 文件 I/O 基本概念

2.9.2 实验内容

掌握 Python 文件 I/O 的概念与用法。

2.9.3 实验原理及步骤

1. open 函数

你必须先用 Python 内置的 open()函数打开一个文件，创建一个 file 对象，相关的方法才可以调用它进行读写。

语法：

file object = open(file_name [, access_mode][, buffering])

各个参数的细节如下：

file_name：file_name 变量是一个包含了你要访问的文件名称的字符串值。

access_mode：access_mode 决定了打开文件的模式：只读，写入，追加等。所有可取值见如下的完全列表。这个参数是非强制的，默认文件访问模式为只读(r)。

buffering:如果 buffering 的值被设为 0，就不会有寄存。如果 buffering 的值取 1，访问文件时会寄存行。如果将 buffering 的值设为大于 1 的整数，表明了这就是的寄存区的缓冲大小。如果取负值，寄存区的缓冲大小则为系统默认。

表 2.9.1 打开文件的不同模式

模式	描述
t	文本模式 (默认)。
x	写模式，新建一个文件，如果该文件已存在则会报错。
b	二进制模式。
+	打开一个文件进行更新(可读可写)。
U	通用换行模式（不推荐）。
r	以只读方式打开文件。文件的指针将会放在文件的开头。这是默认模式。

rb	以二进制格式打开一个文件用于只读。文件指针将会放在文件的开头。这是默认模式。一般用于非文本文件如图片等。
r+	打开一个文件用于读写。文件指针将会放在文件的开头。
rb+	以二进制格式打开一个文件用于读写。文件指针将会放在文件的开头。一般用于非文本文件如图片等。
w	打开一个文件只用于写入。如果该文件已存在则打开文件，并从开头开始编辑，即原有内容会被删除。如果该文件不存在，创建新文件。
wb	以二进制格式打开一个文件只用于写入。如果该文件已存在则打开文件，并从开头开始编辑，即原有内容会被删除。如果该文件不存在，创建新文件。一般用于非文本文件如图片等。
w+	打开一个文件用于读写。如果该文件已存在则打开文件，并从开头开始编辑，即原有内容会被删除。如果该文件不存在，创建新文件。
wb+	以二进制格式打开一个文件用于读写。如果该文件已存在则打开文件，并从开头开始编辑，即原有内容会被删除。如果该文件不存在，创建新文件。一般用于非文本文件如图片等。
a	打开一个文件用于追加。如果该文件已存在，文件指针将会放在文件的结尾。也就是说，新的内容将会被写入到已有内容之后。如果该文件不存在，创建新文件进行写入。
ab	以二进制格式打开一个文件用于追加。如果该文件已存在，文件指针将会放在文件的结尾。也就是说，新的内容将会被写入到已有内容之后。如果该文件不存在，创建新文件进行写入。
a+	打开一个文件用于读写。如果该文件已存在，文件指针将会放在文件的结尾。文件打开时会是追加模式。如果该文件不存在，创建新文件用于读写。
ab+	以二进制格式打开一个文件用于追加。如果该文件已存在，文件指针将会放在文件的结尾。如果该文件不存在，创建新文件用于读写。

2.File 对象的属性

（1）open 方法

一个文件被打开后，你有一个 file 对象，你可以得到有关该文件的各种信息。

以下是和 file 对象相关的所有属性的列表：

表 2.9.2 file 对象相关的所有属性

属性	描述
file.closed	返回 true 如果文件已被关闭，否则返回 false。
file.mode	返回被打开文件的访问模式。
file.name	返回文件的名称。

如下实例：

```
# 打开一个文件
fo = open("foo.txt", "w")
print ("文件名: ", fo.name)
print ("是否已关闭 : ", fo.closed)
print ("访问模式 : ", fo.mode)
```

以上实例输出结果：

文件名: foo.txt

是否已关闭 : False

访问模式 : w

（2）close()方法

File 对象的 close（）方法刷新缓冲区里任何还没写入的信息，并关闭该文件，这之后便不能再进行写入。

当一个文件对象的引用被重新指定给另一个文件时，Python 会关闭之前的文件。用 close（）方法关闭文件是一个很好的习惯。

语法：

fileObject.close()

例子：

```
# 打开一个文件
fo = open("foo.txt", "w")
print ("文件名: ", fo.name)

# 关闭打开的文件
fo.close()
```

以上实例输出结果：

文件名: foo.txt

（3）读写文件：

file 对象提供了一系列方法，能让我们的文件访问更轻松。来看看如何使用 read()和 write()方法来读取和写入文件。

1）write()方法

write()方法可将任何字符串写入一个打开的文件。需要重点注意的是，Python 字符串可以是二进制数据，而不是仅仅是文字。

write()方法不会在字符串的结尾添加换行符('\n')：

语法：

fileObject.write(string)

在这里，被传递的参数是要写入到已打开文件的内容。

例子：

```
# 打开一个文件
fo = open("foo.txt", "w")
fo.write( "www.runoob.com!\nVery good site!\n")

# 关闭打开的文件
fo.close()
```

上述方法会创建 foo.txt 文件，并将收到的内容写入该文件，并最终关闭文件。如果你打开这个文件，将看到以下内容:

```
$ cat foo.txt
hello world!
```

2）read()方法

read（）方法从一个打开的文件中读取一个字符串。需要重点注意的是，Python 字符串可以是二进制数据，而不是仅仅是文字。

语法：

fileObject.read([count])

在这里，被传递的参数是要从已打开文件中读取的字节计数。该方法从文件的开头开始读入，如果没有传入 count，它会尝试尽可能多地读取更多的内容，很可能是直到文件的末尾。

例子：

这里我们用到以上创建的 foo.txt 文件。

```
# 打开一个文件
fo = open("foo.txt", "r+")
```

```
str = fo.read(12)
print ("读取的字符串是 : ", str)
# 关闭打开的文件
fo.close()
```

以上实例输出结果：

读取的字符串是 : hello world!

2.9.4 思考题

Python 的文件 I/O 与 C 语言的文件 I/O 有什么区别？

2.10 Python 进程与线程实验

2.10.1 实验目的

1. 了解 Python 进程与线程的基本概念

2.10.2 实验内容

掌握 Python 进程与线程的概念与用法。

2.10.3 实验原理及步骤

Python 既支持多线程编程，也支持多进程编程。

所谓多进程编程，即将整个程序划分为多个子任务，这些任务在多核 CPU 上可以实现并行执行；反之，在单核 CPU 上，则只能并发执行。在此基础上，我们还可以对每个任务进行更细致地划分，将其分为多个线程，和多进程不同，每个任务的多个线程，只能利用某一个 CPU 并发执行。

几乎所有的操作系统都支持同时运行多个任务，每个任务通常是一个程序，每一个运行中的程序就是一个进程，即进程是应用程序的执行实例。现代的操作系统几乎都支持多进程并发执行。

注意，并发和并行是两个概念，并行指在同一时刻有多条指令在多个处理器上同时执行；并发是指在同一时刻只能有一条指令执行，但多个进程指令被快速轮换执行，使得在宏观上具有多个进程同时执行的效果。

例如，程序员一边开着开发工具在写程序，一边开着参考手册备查，同时还使用电脑播放音乐……除此之外，每台电脑运行时还有大量底层的支撑性程序在运行……这些进程看上去像是在同时工作。

但事实的真相是，对于一个 CPU 而言，在某个时间点它只能执行一个程序。也就是说，只能运行一个进程，CPU 不断地在这些进程之间轮换执行。那么，为什么用户感觉不到任何中断呢？

这是因为相对人的感觉来说，CPU 的执行速度太快了（如果启动的程序足够多，则用户依然可以感觉到程序的运行速度下降了）。所以，虽然 CPU 在多个进程之间轮换执行，但用户感觉到好像有多个进程在同时执行。

线程是进程的组成部分，一个进程可以拥有多个线程。在多线程中，会有一个主线程来完成整个进程从开始到结束的全部操作，而其他的线程会在主线程的运行过程中被创建或退出。

当进程被初始化后，主线程就被创建了，对于绝大多数的应用程序来说，通常仅要求有一个主线程，但也可以在进程内创建多个顺序执行流，这些顺序执行流就是线程。

当一个进程里只有一个线程时，叫作单线程。超过一个线程就叫作多线程。

每个线程必须有自己的父进程，且它可以拥有自己的堆栈、程序计数器和局部变量，但不拥有系统资源，因为它和父进程的其他线程共享该进程所拥有的全部资源。线程可以完成一定的任务，可以与其他线程共享父进程中的共享变量及部分环境，相互之间协同完成进程所要完成的任务。

多个线程共享父进程里的全部资源，会使得编程更加方便，需要注意的是，要确保线程不会妨碍同一进程中的其他线程。

线程是独立运行的，它并不知道进程中是否还有其他线程存在。线程的运行是抢占式的，也就是说，当前运行的线程在任何时候都可能被挂起，以便另外一个线程可以运行。

多线程也是并发执行的，即同一时刻，Python 主程序只允许有一个线程执行，这和全局解释器锁有关系，后续会做详细介绍。

一个线程可以创建和撤销另一个线程，同一个进程中的多个线程之间可以并发运行。

从逻辑的角度来看，多线程存在于一个应用程序中，让一个应用程序可以有多个执行部分同时执行，但操作系统无须将多个线程看作多个独立的应用，对多线程实现调度和管理以及资源分配，线程的调度和管理由进程本身负责完成。

进程和线程的关系为操作系统可以同时执行多个任务，每一个任务就是一个进程，进程可以同时执行多个任务，每一个任务就是一个线程。

程序中通过引入 os 模块，并调用其提供的 fork() 函数，程序中会拥有 2 个进程，其中父进程负责执行整个程序代码，而通过 fork() 函数创建出的子进程，会从创建位置开始，执行后续所有的程序（包含创建子进程的代码）。

fork() 方法的语法格式为：

pid = os.fork()

其中，pid 作为函数的返回值，主进程和子进程都会执行该语句，但主进程执行 fork() 函数得到的 pid 值为非 0 值（其实是子进程的进程 ID），而子进程执行该语句得到的 pid 值为 0。因此，pid 常常作为区分父进程和子进程的标志。

在大多数操作系统中，都会为执行的进程配备唯一的 ID 号，os 模块提供了 getpid() 和 getppid() 函数，可分别用来获取当前进程的 ID 号和父进程的 ID 号。

下面程序演示了 os.fork() 方法的具体使用：

```
import os
print('父进程 ID =', os.getpid())
# 创建一个子进程，下面代码会被两个进程执行
pid = os.fork()
print('当前进程 ID =',os.getpid()," pid=",pid)
#根据 pid 值，分别为子进程和父进程布置任务
if pid == 0:
   print('子进程, ID=',os.getpid()," 父进程 ID=",os.getppid())
else:
   print('父进程, ID=',os.getpid()," pid=",pid)
```

程序输出结果为：

父进程 ID = 2884

当前进程 ID = 2884 pid= 2885

父进程, ID= 2884 pid= 2885

当前进程 ID = 2885 pid= 0

子进程, ID= 2885 父进程 ID= 2884

从输出结果可以看到，当前程序在执行时，系统生成了进程号为 2884 的进程，该进程负责执行当前程序中的所有代码。与此同时，程序第 4 行创建了进程号为 2885 的子进程，该进程将执行第 4 行开始（包括第 4 行）后续的所有代码。

注意，程序第 7 行代码的 if 判断语句，通过判断 pid 值是否为 0，分别为父进程和 fork() 函数创建的子进程布置了不同的执行任务，即子进程负责执行 if 代码块，而父进程则负责执行 else 代码块。

Python 中，有关线程开发的部分被单独封装到了模块中，和线程相关的模块有以下 2 个：

_thread：是 Python 3 以前版本中 thread 模块的重命名，此模块仅提供了低级别的、

原始的线程支持，以及一个简单的锁。功能比较有限。正如它的名字所暗示的（以 _ 开头），一般不建议使用 thread 模块；

threading：Python 3 之后的线程模块，提供了功能丰富的多线程支持，推荐使用。

本节就以 threading 模块为例进行讲解。Python 主要通过两种方式来创建线程：

使用 threading 模块中 Thread 类的构造器创建线程。即直接对类 threading.Thread 进行实例化创建线程，并调用实例化对象的 start() 方法启动线程。

继承 threading 模块中的 Thread 类创建线程类。即用 threading.Thread 派生出一个新的子类，将新建类实例化创建线程，并调用其 start() 方法启动线程。

调用 Thread 类的构造器创建线程

Thread 类提供了如下的 __init__() 构造器，可以用来创建线程：

__init__(self, group=None, target=None, name=None, args=(), kwargs=None, *,daemon=None)

此构造方法中，以上所有参数都是可选参数，即可以使用，也可以忽略。其中各个参数的含义如下：

group：指定所创建的线程隶属于哪个线程组（此参数尚未实现，无需调用）；

target：指定所创建的线程要调度的目标方法（最常用）；

args：以元组的方式，为 target 指定的方法传递参数；

kwargs：以字典的方式，为 target 指定的方法传递参数；

daemon：指定所创建的线程是否为后代线程。

这些参数，初学者只需记住 target、args、kwargs 这 3 个参数的功能即可。

下面程序演示了如何使用 Thread 类的构造方法创建一个线程：

```
import threading
#定义线程要调用的方法，*add 可接收多个以非关键字方式传入的参数
def action(*add):
    for arc in add:
        #调用 getName() 方法获取当前执行该程序的线程名
        print(threading.current_thread().getName() +" "+ arc)
#定义为线程方法传入的参数
```

```
my_tuple = ("http://c.biancheng.net/python/",\
            "http://c.biancheng.net/shell/",\
            "http://c.biancheng.net/java/")
#创建线程
thread = threading.Thread(target = action,args =my_tuple)
thread.start()
```

其输出结果为：

Thread-1 hello world 1

Thread-1 hello world 2

Thread-1 hello world 3

可以看到，新创建的 thread 线程（线程名为 Thread-1）执行了 action() 函数。

2.10.4 思考题

Python 的进程与现成与 C 语言的进程与线程有什么区别？

第3章 Python 项目实验

3.1 九九乘法表实验

3.1.1 实验目的

1. 掌握 Python for 循环及 for 循环嵌套的使用。
2. 掌握 Python print 输出打印信息。

3.1.2 实验内容

通过九九乘法表案例掌握 Python 中 for 循环嵌套的使用以及使用 print 函数输出打印信息。

3.1.3 实验原理及步骤

Python 语言允许在一个循环体里面嵌入另一个循环。而且 for 语句本身就是一条语句，它里面不但可以嵌套 while 语句、if 语句等，其它语句也可以包含在里面。

Python for 循环嵌套语法：

```
for iterating_var in sequence:
  for iterating_var in sequence:
    statements(s)
  statements(s)
```

示例：

```
1x1=1
1x2=2    2x2=4
1x3=3    2x3=6    3x3=9
1x4=4    2x4=8    3x4=12    4x4=16
1x5=5    2x5=10    3x5=15    4x5=20    5x5=25
1x6=6    2x6=12    3x6=18    4x6=24    5x6=30    6x6=36
1x7=7    2x7=14    3x7=21    4x7=28    5x7=35    6x7=42    7x7=49
1x8=8    2x8=16    3x8=24    4x8=32    5x8=40    6x8=48    7x8=56    8x8=64
1x9=9    2x9=18    3x9=27    4x9=36    5x9=45    6x9=54    7x9=63    8x9=72    9x9=81
```

图 3.1.1 九九乘法表

案例分析：

因为 Python 的 print 函数只能按行输出，所以通过分析上图九九乘法表可发现其规律：随着行数的增加每行的算式数也随之增加，假设其行数为 N，则该行算式数为 N；每行算式 a*b=c 的第一位 a 均从 1 开始递增最大不超过该行数 N，第二位 b 是固定的且等于其行数 N。

由上述分析可知，此案例需要一个循环套嵌的方式实现：最外层循环用来实现行数的控制所以需要遍历从 1 到 9；内部循环需要用以控制算式第一位 a 的变换，即从 1 开始递增且小于等于该行行数。所以代码实现如下所示：

```
# 把 1-9 的数值依次赋值给 num；右边为 range
for num in range(1, 10):
    # 把 range(1,num+1)赋值给 x；当 num = 1，x=(1,2),此时 x 的取值就是 1；
    for x in range(1, num + 1):
        # 假如 num=1,x=1，这句的意思是 1*1=(1*1),end=' '表示不换行以空格为分隔符,还可以用\t 制表符方式分割;
        print('%d*%d=%2d' % (x, num , num * x), end=' ')
    # print()表示换行
    print()
```

3.1.4 思考题

使用 while 循环完成九九乘法表。

3.2 闰年判断实验

3.2.1 实验目的

1. 掌握 Python 条件判断语句的使用。
2. 掌握闰年判断的方法。

3.2.2 实验内容

通过闰年判断案例掌握 Python 中条件判断语句的使用以及闰年判断的方法。

3.2.3 实验原理及步骤

闰年（Leap Year）是为了弥补因人为历法规定造成的年度天数与地球实际公转周期的时间差而设立的。补上时间差的年份为闰年。闰年共有 366 天（1 月~12 月分别为 31 天、29 天、31 天、30 天、31 天、30 天、31 天、31 天、30 天、31 天、30 天、31 天）。

闰年（公历中的名词）和闰月（农历中的名词）并没有直接的关联，公历只分闰年和平年，平年有 365 天，闰年有 366 天（2 月中多一天）；平年中也可能有闰月（如 2017 年是平年，农历有闰月，闰六月）。

闰年是历法中的名词，分为普通闰年和世纪闰年。

普通闰年：公历年份是 4 的倍数，且不是 100 的倍数的，为闰年（如 2004 年、2020 年等就是闰年）。

世纪闰年：公历年份是整百数的，必须是 400 的倍数才是闰年（如 1900 年不是闰年，2000 年是闰年）。

示例代码如下：

```
year = input("请输入您要判断的年份：")
year = int(year)
result = (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0)
if result:
    s = "是"
else:
    s = "不是"
print("{0}年{1}闰年".format(year, s))
```

3.2.4 思考题

改进代码以实现多次判断以及同时输入多个年份进行判断。

3.3 数字组合实验

3.3.1 实验目的

1. 掌握 Python 字符串的遍历拆分与组合。
2. 掌握 Python 列表遍历方法。
3. 掌握 Python 链表推导式。

3.3.2 实验内容

通过数字组合的案例掌握 Python 中字符串的遍历拆分与组合、列表的循环遍历以及掌握 Python 链表推导式。

3.3.3 实验原理及步骤

字符串的遍历拆分与组合：

在 Python 中字符串属于可迭代对象，所以字符串的遍历拆分可通过 for 循环的方式拆分。例如：

```
a = "12345678"
for num in a:
    print(num, end=" ")
```

其运行结果为：

```
1 2 3 4 5 6 7 8
```

列表与字符串同样属于可迭代对象，所以其循环遍历也可以用 for 的方式实现，这里就不做过多演示。

python 中的链表推导式：

链表推导式又称列表推导式（list comprehension）是一种方便简单的语法形式，可以利用它将一个列表经过过滤后转换成另一个列表，也可以利用它将函数应用与列表中的元素。其书写形式如下：

```
[表达式 for 变量 in 列表 ]
或者
[表达式 for 变量 in 列表 if 条件]
```

链表推导式提供了一个创建链表的简单途径，无需使用 map()， filter() 以及 lambda。 返回链表的定义通常要比创建这些链表更清晰。每一个链表推导式包括在一个 for 语句之后的表达式，零或多个 for 或 if 语句。返回值是由 for 或 if 子句之后的表

达式得到的元素组成的链表。例如返回由 10 以内所有素数组成的列表：

```
list1 = [i for i in range(10) if i %2 !=0 ]
print(list1)

[1, 3, 5, 7, 9]
```

案例分析：

案例要求输入两组数字，并对其进行拆分后两两组合，并输出其组合数。

由上述分析可知，实现数字组合案例，需要先键盘输入两组数值数值，并在拆分后进行组合并技术。演示代码如下：

```
num1 = input('请输入 0-9 之间任意一个或多个数字：')
num2 = input('请再次输入 0-9 之间任意一个或多个数字：')



def account(num1, num2):
    print("两次输入数字的组合即将开始")
    list1 = [(x, y) for x in num1 for y in num2]
    num3 = []
    for list2 in list1:
        print(".join([str(x) for x in list2]))
        num3.append(".join([str(x) for x in list2]))
    print("组合完成，共有" + str(len(num3)) + "对组合！")



account(num1, num2)
```

3.3.4 思考题

完善代码使其更具有鲁棒性，比如实现判断数据格式是否符合规则若不符合请给出提示信息，尝试实现三组四组数据的组合排列。

3.4 Python web 服务器实验

3.4.1 实验目的

1. 了解什么是 web 服务器。
2. 掌握 Python 实现简单的 web 服务器。

3.4.2 实验内容

通过 Python web 服务器案例了解什么是 web 服务器，掌握 Python 实现简单的 web 服务器。

3.4.3 实验原理及步骤

“web 服务器”负责响应来自用户端比如浏览器的请求，并向客户端返回静态资源的网页，比如图片，视频，网盘上分享的各种文件下载等。Web 服务器只处理静态的文件而不处理动态内容，仅接受和完成 HTTP 超文本传输协议的请求。目前流行的用来搭建 Web 服务可选软件有 Apache，Nginx 及微软的 IIS 等。

简单的说，web 服务器就是机器上监听了本地的 IP 和端口的一个进程，当有客户的请求进来时，它会响应客户的请求并返回相应的信息。客户端和服务端之间的通信基于 HTTP 协议，客户端可以是浏览器或者任何支持 HTTP 的应用。我们平时访问网页或手机 APP 里接受服务端的数据，都是通过这样的方式。

下面这张图是一个客户端和服务端交互的过程：

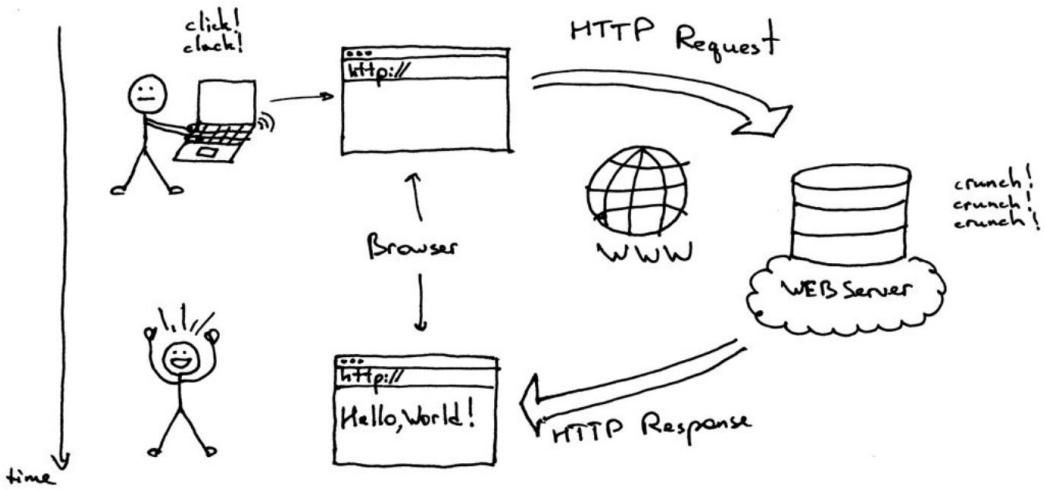

图 3.4.1 客户端和服务端交互的过程

用户在浏览器输入网址向 web 服务器发出一个请求，web 服务器收到请求后返回 Hello, world!给浏览器，用户就能在浏览器上看到文字。

我们来用 Python 实现这样一个简单的 web 服务器。代码如下：

```
#!/usr/bin/python
# _*_ coding: UTF-8 _*_
from socket import *
import threading
import os

'''自定义线程函数'''
def Server(tcpClisock, addr):

    BUFSIZE = 1024
    print('Waiting for the connection：', addr)
    data = tcpClisock.recv(BUFSIZE).decode()
    filename = data.split()[1]
    filename = filename[1:]

    '''当网络质量差没有收到浏览器的访问数据时执行'''
    if filename == '':
        tcpClisock.close()
        print("请输入要访问的文件")

    base_dir = os.getcwd()
    file_dir = os.path.join(base_dir,filename)

    '''当访问的文件在本地服务器存在时执行'''
    if os.path.exists(file_dir):
        f = open(file_dir,encoding = 'utf-8')
        SUCCESS_PAGE = "HTTP/1.1 200 OK\r\n\r\n" + f.read()
        print(SUCCESS_PAGE)
        tcpClisock.sendall(SUCCESS_PAGE.encode())
        tcpClisock.close()
    else:
        FAIL_PAGE = "HTTP/1.1 404 NotFound\r\n\r\n" + open(os.path.join(base_dir, "fail.html"),
encoding="utf-8").read()
        print(FAIL_PAGE)
        tcpClisock.sendall(FAIL_PAGE.encode())
        tcpClisock.close()
```

```
'''主函数'''
if __name__ == '__main__':

    '''分配 IP、端口、创建套接字对象'''
    ADDR = ("", 8080)
    tcpSersock = socket(AF_INET, SOCK_STREAM)
    tcpSersock.bind(ADDR)
    tcpSersock.listen(5)
    print("waiting for connection.....\n")
    while True:
        tcpClisock, addr = tcpSersock.accept()
        thread = threading.Thread(target=Server, args=(tcpClisock, addr))
        thread.start()
    tcpSersock.close()
```

程序运行后在浏览器地址栏中输入 http://localhost:8080/success.html 或 http://localhost:8080/fail.html 查看服务器返回信息。

3.4.4 思考题

查询资料使用其它 web 框架（如 flask、Django）搭建 web 服务器。

3.5 Python 爬虫实验

3.5.1 实验目的

1. 掌握 Python Requests 库的基本使用。
2. 了解爬虫机制原理。

3.5.2 实验内容

通过爬取有道在线翻译的案例掌握 Python Requests 库的基本使用，了解爬虫机制原理。

3.5.3 实验原理及步骤

Requests 是用 Python 语言编写，基于 urllib，采用 Apache2 Licensed 开源协议的 HTTP 库。它比 urllib 更加方便，可以节约我们大量的工作，完全满足 HTTP 测试需求。

使用 Requests 库获取某个网页：

```
import requests
r = requests.get("https://api.github.com/events")
print(r)            # <Response [200]>
print(type(r))      # <class 'requests.models.Response'>
print(r.status_code)   # 200
```

各种请求：

```
# 发送一个 HTTP POST 请求：
r = requests.post("http://httpbin.org/post",data = {'key':'value'})
r = requests.delete('http://httpbin.org/delete')   # 发送一个 HTTP delete 请求：
r = requests.head('http://httpbin.org/get')       # 发送一个 HTTP head 请求：
r = requests.options('http://httpbin.org/get')     # 发送一个 HTTP options 请求：
```

网络爬虫（又称为网页蜘蛛，网络机器人，在 FOAF 社区中间，更经常的称为网页追逐者），是一种按照一定的规则，自动地抓取万维网信息的程序或者脚本。另外一些不常使用的名字还有蚂蚁、自动索引、模拟程序或者蠕虫。

简单来讲，爬虫就是一个探测机器，它的基本操作就是模拟人的行为去各个网站溜达，点点按钮，查查数据，或者把看到的信息下载回来。

如果要爬取 web 端的有道翻译，则必须要获取到他的接口：首先打开有道翻译的链接：http://fanyi.youdao.com/。然后在页面中右键->检查->Network 项。这时候就来到了网络监听窗口，以后你在这个页面中发送的所有网络请求，都会在 Network 这个地

方显示出来。接着我们在翻译的窗口输入我们需要翻译的文字，比如输入 hello。然后点击自动翻译按钮，那么接下来在下面就可以看到浏览器给有道发送的请求，这里截个图看看：

图 3.5.1 查看网络请求

在上图，我们可以看到发送了很多的网络请求，这里我们点击第一个网络请求进行查看：

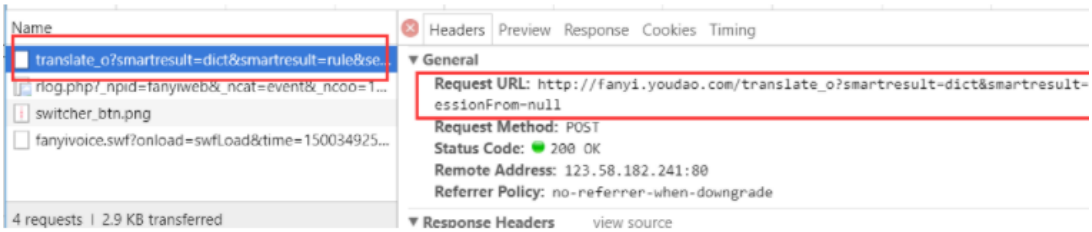

图 3.5.2 查看具体网络请求

可以看到，我们在点击自动翻译的时候，发送的请求就是上图中 Request URL 的那个 URL，然后我们再点击那个 Response，我们可以看到返回的结果：

图 3.5.3 查看返回结果

现在我们再回到 Headers 的地方，然后滚动到最下面，可以看到有一个 Form Data 的地方，这个下面展示了许多的数据，这些数据就是你在点击翻译的时候浏览器给服

务器发送的数据：

▼ **Form Data** view source view URL encoded
i: hello
from: AUTO
to: AUTO
smartresult: dict
client: fanyideskweb
salt: 1500349255670
sign: 997742c66698b25b43a3a5030e1c2ff2
doctype: json
version: 2.1
keyfrom: fanyi.web
action: FY_BY_CL1CKBUTTON
typoResult: true

图 3.5.4 发送的数据

到现在为止，我们就可以写一个简单的爬虫，去调用有道翻译的接口了：

```
#！ /usr/bin evn python3
# -*- coding:utf-8 -*-
import requests


def youdao_translate(value):
    if value == '':
        print('输入内容为空@_@')
        return False
    else:

        # Request URL
        url = 'http://fanyi.youdao.com/translate?smartresult=dict&smartresult=rule'

        # 待提交准备 Post 给 url 的 Data:定义为 dict
        data = {
            'i': value
            , 'from': 'AUTO'
            , 'to': 'AUTO'
            , 'smartresult': 'dict'
            , 'client': 'fanyideskweb'
            , 'salt': '15450970848938'
            , 'sign': '1013a7a1416565e32eb3092d219eebf2'
            , 'ts': '1545097084893'
            , 'bv': 'b33a2f3f9d09bde064c9275bcb33d94e'
            , 'doctype': 'json'
            , 'version': '2.1'
            , 'keyfrom': 'fanyi.web'
```

```
        , 'action': 'FY_BY_REALTIME'
        , 'typoResult': 'false'
    }
    # 使用 urlencode 方法转换标准格式
    r = requests.post(url, data=data)
    # print(r.url);
    target = r.json();
    print("翻译结果：")
    print(target['translateResult'][0][0]['tgt'])
    print()
    return True


if __name__ == '__main__':
    try:
        while True:
            word = input('请输入待翻译的单词或句子:\n').strip()
            youdao_translate(word)
    except KeyboardInterrupt:
        print('\a 手动退出!欢迎再来')
```

3.5.4 思考题

查询资料了解爬虫与反爬虫机制，并尝试爬取其它网站信息。

3.6 Python 飞机大战实验

3.6.1 实验目的

1. 了解 Python pygame 包的安装与使用。
2. 掌握 Python 飞机大战游戏开发。

3.6.2 实验内容

掌握 Python 飞机大战游戏综合案例开发。

3.6.3 实验原理及步骤

1. 游戏设定

敌方共有大中小 3 款飞机，分为高中低三种速度;子弹的射程并非全屏,而大概是屏幕长度的 80%;消灭小飞机需要 1 发子弹,中飞机需要 8 发,大飞机需要 20 发子弹;每消灭一架小飞机得 1000 分,中飞机 6000 分,大飞机 10000 分;每隔 30 秒有一个随机的道具补给,分为两种道具，全屏炸弹和双倍子弹;全屏炸弹最多只能存放 3 枚,双倍子弹可以维持 18 秒钟的效果;游戏将根据分数来逐步提高难度,难度的提高表现为飞机数量的增多以及速度的加快。另外还对游戏做了一些改进,比如为中飞机和大飞机增加了血槽的显示,这样玩家可以直观地知道敌机快被消灭了没有;我方有三次机会,每次被敌人消灭,新诞生的飞机会有 3 秒钟的安全期;游戏结束后会显示历史最高分数。

图 3.6.1 飞机大战

2. 我方飞机

首先创建一个 myplane. py 模块来定义我方飞机:

```
import pygame


class MyPlane(pygame.sprite.Sprite):
    def __init__(self, bg_size):
        pygame.sprite.Sprite.__init__(self)


        self.image1 = pygame.image.load("images/me1.png").convert_alpha()
        self.image2 = pygame.image.load("images/me2.png").convert_alpha()
        self.destroy_images = []
        self.destroy_images.extend([\
            pygame.image.load("images/me_destroy_1.png").convert_alpha(), \
            pygame.image.load("images/me_destroy_2.png").convert_alpha(), \
            pygame.image.load("images/me_destroy_3.png").convert_alpha(), \
            pygame.image.load("images/me_destroy_4.png").convert_alpha() \
            ])
        self.rect = self.image1.get_rect()
        self.width, self.height = bg_size[0], bg_size[1]
        self.rect.left, self.rect.top = \
                    (self.width - self.rect.width) // 2, \
                    self.height - self.rect.height - 60
        self.speed = 10
        self.active = True
```

```
        self.invincible = False
        self.mask = pygame.mask.from_surface(self.image1)

    # 分别定义 moveUp()、moveDown()、moveLeft()和 moveRight()控制我方飞机上、下、左、右移动:

    def moveUp(self):
        if self.rect.top > 0:
            self.rect.top -= self.speed
        else:
            self.rect.top = 0

    def moveDown(self):
        if self.rect.bottom < self.height - 60:
            self.rect.top += self.speed
        else:
            self.rect.bottom = self.height - 60

    def moveLeft(self):
        if self.rect.left > 0:
            self.rect.left -= self.speed
        else:
            self.rect.left = 0
```

```
    def moveRight(self):
        if self.rect.right < self.width:
            self.rect.left += self.speed
        else:
            self.rect.right = self.width


    def reset(self):
        self.rect.left, self.rect.top = \
                    (self.width - self.rect.width) // 2, \
                    self.height - self.rect.height - 60
        self.active = True
        self.invincible = True
```

3. 敌方飞机

既然英雄已经有了,那现在就是需要创造敌人的时候。敌机分为小、中、大三个尺寸,它们的速度依次是快、中、慢,在游戏界面的上方位置创造位置随机的敌机,可以让它们不在同一排出现。将敌机的定义写在 enemy. py 模块中:

```
import pygame
from random import *



class SmallEnemy(pygame.sprite.Sprite):
    def __init__(self, bg_size):
        pygame.sprite.Sprite.__init__(self)
```

```
        self.image = pygame.image.load("images/enemy1.png").convert_alpha()
        self.destroy_images = []
        self.destroy_images.extend([ \
            pygame.image.load("images/enemy1_down1.png").convert_alpha(), \
            pygame.image.load("images/enemy1_down2.png").convert_alpha(), \
            pygame.image.load("images/enemy1_down3.png").convert_alpha(), \
            pygame.image.load("images/enemy1_down4.png").convert_alpha() \
            ])
        self.rect = self.image.get_rect()
        self.width, self.height = bg_size[0], bg_size[1]
        self.speed = 2
        self.active = True
        self.rect.left, self.rect.top = \
            randint(0, self.width - self.rect.width), \
            randint(-5 * self.height, 0)
        self.mask = pygame.mask.from_surface(self.image)

    def move(self):
        if self.rect.top < self.height:
            self.rect.top += self.speed
        else:
            self.reset()

    def reset(self):
```

```
        self.active = True
        self.rect.left, self.rect.top = \
            randint(0, self.width - self.rect.width), \
            randint(-5 * self.height, 0)



class MidEnemy(pygame.sprite.Sprite):
    energy = 8

    def __init__(self, bg_size):
        pygame.sprite.Sprite.__init__(self)

        self.image = pygame.image.load("images/enemy2.png").convert_alpha()
        self.image_hit = pygame.image.load("images/enemy2_hit.png").convert_alpha()
        self.destroy_images = []
        self.destroy_images.extend([ \
            pygame.image.load("images/enemy2_down1.png").convert_alpha(), \
            pygame.image.load("images/enemy2_down2.png").convert_alpha(), \
            pygame.image.load("images/enemy2_down3.png").convert_alpha(), \
            pygame.image.load("images/enemy2_down4.png").convert_alpha() \
            ])
        self.rect = self.image.get_rect()
        self.width, self.height = bg_size[0], bg_size[1]
        self.speed = 1
```

```
        self.active = True
        self.rect.left, self.rect.top = \
            randint(0, self.width - self.rect.width), \
            randint(-10 * self.height, -self.height)
        self.mask = pygame.mask.from_surface(self.image)
        self.energy = MidEnemy.energy
        self.hit = False

    def move(self):
        if self.rect.top < self.height:
            self.rect.top += self.speed
        else:
            self.reset()

    def reset(self):
        self.active = True
        self.energy = MidEnemy.energy
        self.rect.left, self.rect.top = \
            randint(0, self.width - self.rect.width), \
            randint(-10 * self.height, -self.height)


class BigEnemy(pygame.sprite.Sprite):
    energy = 20
```

```
    def __init__(self, bg_size):
        pygame.sprite.Sprite.__init__(self)


        self.image1 = pygame.image.load("images/enemy3_n1.png").convert_alpha()
        self.image2 = pygame.image.load("images/enemy3_n2.png").convert_alpha()
        self.image_hit = pygame.image.load("images/enemy3_hit.png").convert_alpha()
        self.destroy_images = []
        self.destroy_images.extend([ \
            pygame.image.load("images/enemy3_down1.png").convert_alpha(), \
            pygame.image.load("images/enemy3_down2.png").convert_alpha(), \
            pygame.image.load("images/enemy3_down3.png").convert_alpha(), \
            pygame.image.load("images/enemy3_down4.png").convert_alpha(), \
            pygame.image.load("images/enemy3_down5.png").convert_alpha(), \
            pygame.image.load("images/enemy3_down6.png").convert_alpha() \
            ])
        self.rect = self.image1.get_rect()
        self.width, self.height = bg_size[0], bg_size[1]
        self.speed = 1
        self.active = True
        self.rect.left, self.rect.top = \
            randint(0, self.width - self.rect.width), \
            randint(-15 * self.height, -5 * self.height)
        self.mask = pygame.mask.from_surface(self.image1)
        self.energy = BigEnemy.energy
```

```
        self.hit = False


    def move(self):
        if self.rect.top < self.height:
            self.rect.top += self.speed
        else:
            self.reset()


    def reset(self):
        self.active = True
        self.energy = BigEnemy.energy
        self.rect.left, self.rect.top = \
            randint(0, self.width - self.rect.width), \
            randint(-15 * self.height, -5 * self.height)
```

4. 发射子弹

现在的情况是我方飞机处于落后挨打的状态,敌强我弱,所以应该拿起武器进行反击!接下来定义子弹,子弹分为两种: 一种是普通子弹一次只发射一颗; 另一种是补给发放的超级子弹一次可以发射两颗。

将子弹定义为独立的模块 bullet.py:

```
import pygame




class Bullet1(pygame.sprite.Sprite):
    def __init__(self, position):
        pygame.sprite.Sprite.__init__(self)
```

```
        self.image = pygame.image.load("images/bullet1.png").convert_alpha()
        self.rect = self.image.get_rect()
        self.rect.left, self.rect.top = position
        self.speed = 12
        self.active = False
        self.mask = pygame.mask.from_surface(self.image)

    def move(self):
        self.rect.top -= self.speed

        if self.rect.top < 0:
            self.active = False

    def reset(self, position):
        self.rect.left, self.rect.top = position
        self.active = True



class Bullet2(pygame.sprite.Sprite):
    def __init__(self, position):
        pygame.sprite.Sprite.__init__(self)

        self.image = pygame.image.load("images/bullet2.png").convert_alpha()
```

```
        self.rect = self.image.get_rect()
        self.rect.left, self.rect.top = position
        self.speed = 14
        self.active = False
        self.mask = pygame.mask.from_surface(self.image)

    def move(self):
        self.rect.top -= self.speed

        if self.rect.top < 0:
            self.active = False

    def reset(self, position):
        self.rect.left, self.rect.top = position
        self.active = True
```

5. 发放补给包

游戏设计每 30 秒随机发放一个补给包,可能是超级子弹,也可能是全屏炸弹。补给包有自己的图像和运动轨迹,不妨单独为其定义一个模块 supply.py:

```
import pygame
from random import *

class Bullet_Supply(pygame.sprite.Sprite):
    def __init__(self, bg_size):
```

```
        pygame.sprite.Sprite.__init__(self)


        self.image = pygame.image.load("images/bullet_supply.png").convert_alpha()
        self.rect = self.image.get_rect()
        self.width, self.height = bg_size[0], bg_size[1]
        self.rect.left, self.rect.bottom = \
            randint(0, self.width - self.rect.width), -100
        self.speed = 5
        self.active = False
        self.mask = pygame.mask.from_surface(self.image)


    def move(self):
        if self.rect.top < self.height:
            self.rect.top += self.speed
        else:
            self.active = False


    def reset(self):
        self.active = True
        self.rect.left, self.rect.bottom = \
            randint(0, self.width - self.rect.width), -100



class Bomb_Supply(pygame.sprite.Sprite):
```

```
    def __init__(self, bg_size):
        pygame.sprite.Sprite.__init__(self)

        self.image = pygame.image.load("images/bomb_supply.png").convert_alpha()
        self.rect = self.image.get_rect()
        self.width, self.height = bg_size[0], bg_size[1]
        self.rect.left, self.rect.bottom = \
            randint(0, self.width - self.rect.width), -100
        self.speed = 5
        self.active = False
        self.mask = pygame.mask.from_surface(self.image)

    def move(self):
        if self.rect.top < self.height:
            self.rect.top += self.speed
        else:
            self.active = False

    def reset(self):
        self.active = True
        self.rect.left, self.rect.bottom = \
            randint(0, self.width - self.rect.width), -100
```

6. 主模块

所有的模块应该放在同一个文件夹下，最后运行程序就如下图所示。

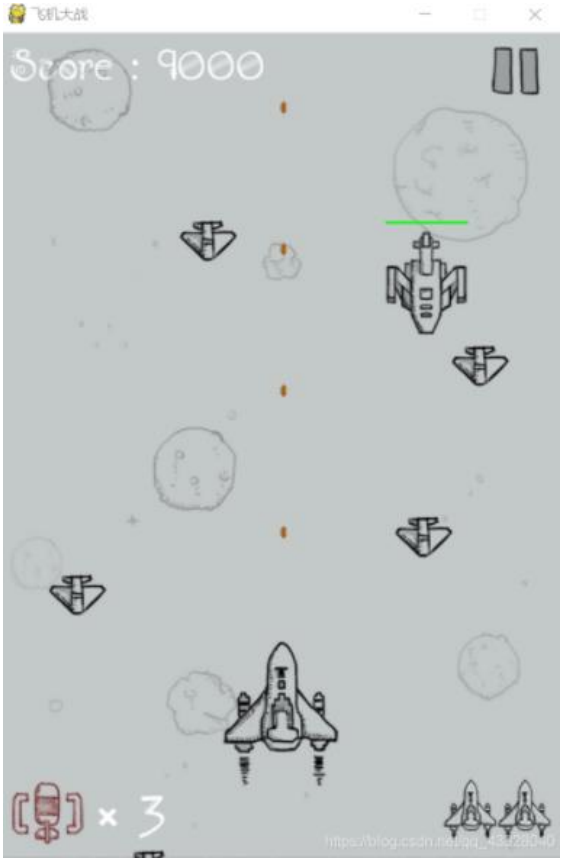

图 3.6.2 飞机大战

3.6.4 思考题

使用 pygame 包自己动手完成游戏开发。